

Block 1: Introduction to Algorithms and Data Structures

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 1

Analysis of Algorithms

ഒരു algorithm എത്ര സമയവും (time) എത്ര memory-യും (space) എടുക്കുന്നു എന്ന് input size n -ന്റെ function ആയി കണക്കാക്കുന്നതാണ് ഈ unit. Big O, Ω , Θ notations, step counting, insertion sort-ന്റെ best/worst/average case analysis, space complexity, complexity ladder എന്നിവ ഇവിടെ പഠിക്കുന്നു.

Algorithm and its five characteristics

Complexity classes P and NP

Complexity, asymptotic analysis and the space-time tradeoff

Θ -notation (tight bound)

Big O notation (upper bound)

Ω -notation (lower bound)

Simplifying with asymptotic notation and rules of big O

Process of analysis: performance vs complexity, the RAM model

Analysing statements: sequence, if-else, loops, nested loops

Counting steps: the partial-sum function

Exact analysis of insertion sort; best, worst and average case

Calculation of storage (space) complexity

Worked example: GCD by repeated subtraction

Calculation of time complexity: constant code, binary search, TSP

The complexity ladder and comparison tables

Algorithm and its five characteristics

Algorithm: a finite sequence of definite and effective instructions which, for a given input, terminates after producing the correct output. **Program:** an algorithm coded in a specific computer language.

Algorithm: ഒരു input-ന് correct output ഉണ്ടാക്കിയ ശേഷം അവസാനിക്കുന്ന, definite-ഉം effective-ഉം ആയ instructions-ന്റെ finite sequence. **Program:** ഒരു പ്രത്യേക computer language-ൽ എഴുതിയ algorithm.

A computer can only carry out predefined instructions. The formal step-by-step description of how to solve a problem is the algorithm; writing it in C gives a program. IGNOU lists five features every algorithm must have: input, output, definiteness, effectiveness and termination. Even with very fast computers we still need time-efficient algorithms, because a bad algorithm on a big input can take years.

മലയാളത്തിൽ

Algorithm എന്നത് ഒരു പാചകക്കുറിപ്പ് (recipe) പോലെയാണ്: ചേരുവകൾ (input), ഓരോ step-ഉം വ്യക്തമായി (definiteness), ചെയ്യാൻ പറ്റുന്ന കാര്യങ്ങൾ മാത്രം (effectiveness), ഒടുവിൽ പായസം റെഡി (output), പാചകം ഒരിക്കൽ തീരണം (termination). ഇതേ recipe C language-ൽ എഴുതിയാൽ അതാണ് program.

Computer എത്ര വേഗമുള്ളതായാലും മോശം algorithm ആണെങ്കിൽ വലിയ input-ന് മണിക്കൂറുകളോ വർഷങ്ങളോ എടുക്കും.

1. Input: zero or more quantities supplied from outside.
2. Output: at least one quantity is produced.
3. Definiteness: each instruction is clear and unambiguous.
4. Effectiveness: each instruction is basic enough to be carried out exactly, in finite time.
5. Termination: the algorithm stops after a finite number of steps.

EXAMPLE

Is "Step 1: $i = 1$. Step 2: print i . Step 3: go to Step 2." an algorithm?

It has no real input issue and each step is definite and effective, but it never stops: it prints 1 forever. It fails **termination**, so it is not an algorithm.

EXAMPLE

Why is "Step 2: add a suitable number to x " not acceptable?

"Suitable" is not defined, so two people could do different things. The step fails **definiteness**.

Exam tip: "Define algorithm and list its characteristics" is a common 5-mark question. Write the definition, then the five features with one line each.

Complexity classes P and NP

Complexity class: a set of decision problems of comparable resource requirements. **P:** decision problems solvable by a deterministic machine in polynomial time. **NP:** decision problems solvable by a non-deterministic machine in polynomial time.

Complexity class: ഏകദേശം ഒരേ resource ആവശ്യമുള്ള decision problems-ന്റെ set. **P:** deterministic machine-ന് polynomial time-ൽ solve ചെയ്യാവുന്ന decision problems. **NP:** non-deterministic machine-ന് polynomial time-ൽ solve ചെയ്യാവുന്ന decision problems.

Complexity theory studies the resources (mainly time and space) needed to solve a problem. It is different from computability theory, which only asks whether a problem can be solved by any algorithm at all. A decision problem is one with a yes/no answer. Problems in P are considered efficiently solvable; this unit analyses such algorithms. NP contains problems like Boolean satisfiability, Hamiltonian path and vertex cover.

മലയാളത്തിൽ

Computability theory ചോദിക്കുന്നത് "ഈ problem solve ചെയ്യാൻ പറ്റുമോ?" എന്ന് മാത്രം; complexity theory ചോദിക്കുന്നത് "എത്ര സമയം, എത്ര memory വേണം?" എന്നാണ്. P-യിലെ problems polynomial time-ൽ (n , n^2 , n^3 പോലെ) തീർക്കാം, അതുകൊണ്ട് practical ആണ്. NP-യിലെ problems-ന് ഒരു ഉത്തരം തന്നാൽ അത് ശരിയാണോ എന്ന് polynomial time-ൽ check ചെയ്യാം; പക്ഷേ SAT പോലുള്ള പലതിനും ഉത്തരം വേഗം കണ്ടുപിടിക്കാനുള്ള വഴി ഇതുവരെ ആർക്കും അറിയില്ല.

$P \subseteq NP$ (every deterministic machine is a special non-deterministic machine)

Whether $P = NP$ is still an open question.

Examples in NP: SAT, Hamiltonian path, vertex cover

EXAMPLE

"Is the number x present in this array of n numbers?" Which class?

Linear search answers it in at most n comparisons, i.e. polynomial (linear) time on a deterministic machine. So it is in P (and hence also in NP).

Exam tip: Say "decision problem" in both definitions, and remember the only difference is deterministic (P) vs non-deterministic (NP) machine.

Complexity, asymptotic analysis and the space-time tradeoff

Complexity: the rate at which the required storage or time grows as a function of the problem size.

Asymptotic analysis: describing that growth, for large n , by proportionality to a known function, independent of machine and compiler.

Complexity: problem size കൂടുമ്പോൾ ആവശ്യമായ storage അല്ലെങ്കിൽ time വളരുന്ന നിരക്ക് (rate), size-ന്റെ function ആയി. **Asymptotic analysis:** വലിയ n -ന് ആ വളർച്ചയെ ഒരു അറിയപ്പെടുന്ന function-നോടുള്ള proportionality ആയി, machine-ഉം compiler-ഉം നോക്കാതെ, വിവരിക്കുന്നത്.

The exact seconds a program takes depend on the machine, compiler and many other factors. So we do not measure absolute time; we ask how time or memory grows when n grows, and compare with a simple function such as n , n^2 or $\log n$. This is asymptotic analysis, expressed with O , Ω and Θ . Note that we analyse an algorithm, not a problem: a simple problem can have a slow algorithm. Sometimes we trade space for time, e.g. using a big table to avoid recomputation; complexity analysis gives a verifiable basis for such choices.

മലയാളത്തിൽ

ഒരേ program ഒരു പഴയ laptop-ലും ഒരു പുതിയ server-ലും ഓടിച്ചാൽ സമയം വ്യത്യസ്തമാകും; അതിനാൽ seconds അല്ല നോക്കുന്നത്. n ഇരട്ടിച്ചാൽ സമയം ഇരട്ടിക്കുമോ, നാലിരട്ടിയാകുമോ എന്നതാണ് പ്രധാനം. ഇതാണ് asymptotic analysis. Space-time tradeoff: പരീക്ഷയ്ക്ക് മുമ്പ് എല്ലാ formula-കളും ഒരു chart-ൽ എഴുതി വെച്ചാൽ (കൂടുതൽ space) ഓരോ തവണയും derive ചെയ്യേണ്ട സമയം ലാഭിക്കാം.

Resources: time (number of steps) and space (memory)

Asymptotic notations: O (upper bound), Ω (lower bound), Θ (tight bound)

Little-o is similar in spirit to big O but rarely used in this course

EXAMPLE

Algorithm A takes $100n$ steps and algorithm B takes n^2 steps. Which is better for large n ?

For $n = 50$: $A = 5000$, $B = 2500$ (B faster). For $n = 100$: both 10,000. For $n = 1000$: $A = 100,000$, $B = 1,000,000$. For every $n > 100$, A is faster. Asymptotically A is $O(n)$ and B is $O(n^2)$, so A is better for large inputs.

EXAMPLE (SPACE-TIME TRADEOFF)

We must answer many queries "sum of $A[0..k]$?" on a fixed array of n numbers.

Without extra memory each query adds up to n numbers: $O(n)$ time, $O(1)$ extra space. If we store a prefix-sum array $P[k] = A[0] + \dots + A[k]$ once, each query is one lookup: $O(1)$ time, but $O(n)$ extra space.

Θ -notation (tight bound)

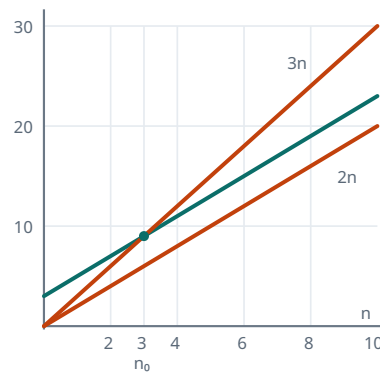
Θ -notation: $f(n) = \Theta(g(n))$ if there exist positive constants c_1 , c_2 and n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$; $g(n)$ is an asymptotically tight bound for $f(n)$.

Θ -notation: എല്ലാ $n \geq n_0$ -നും $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ ആകുന്ന positive constants c_1 , c_2 , n_0 ഉണ്ടെങ്കിൽ $f(n) = \Theta(g(n))$; അപ്പോൾ $g(n)$, $f(n)$ -ന്റെ asymptotically tight bound ആണ്.

Θ traps $f(n)$ between two constant multiples of $g(n)$ from some point n_0 onwards. So f and g grow at the same rate, up to constant factors. To prove $f(n) = \Theta(g(n))$ you must actually find c_1 , c_2 and n_0 ; to disprove it, show that no constant can work for large n .

മലയാളത്തിൽ

Θ എന്നാൽ $f(n)$ -നെ രണ്ട് "മതിലുകൾക്ക്" ഇടയിൽ പിടിച്ചിടുന്നതാണ്: താഴെ $c_1 g(n)$, മുകളിൽ $c_2 g(n)$. n_0 -ന് ശേഷം $f(n)$ ഒരിക്കലും ഈ band-ന് പുറത്ത് പോകില്ല. അതായത് f -ഉം g -ഉം ഒരേ വേഗത്തിൽ വളരുന്നു; ബസ്സും അതിന് പിന്നാലെ വരുന്ന ബൈക്കും ഒരേ speed-ൽ പോകുന്നത് പോലെ.



Top line $3n$, bottom line $2n$, middle line $f(n) = 2n + 3$. $f(n)$ lies between $c_1g(n) = 2n$ and $c_2g(n) = 3n$ for all $n \geq n_0 = 3$, so $2n + 3 = \Theta(n)$. At $n = 3$ the curves f and $3n$ meet (both 9).

EXAMPLE (FROM THE UNIT)

Show that $(1/3)n^2 - 4n = \Theta(n^2)$.

We need $c_1n^2 \leq (1/3)n^2 - 4n \leq c_2n^2$ for all $n \geq n_0$. Divide by n^2 : $c_1 \leq 1/3 - 4/n \leq c_2$.

Right-hand inequality: $1/3 - 4/n < 1/3$ for every $n \geq 1$, so $c_2 = 1/3$ works.

Left-hand inequality: $1/3 - 4/n$ grows with n . At $n = 13$ it is $1/3 - 4/13 = (13 - 12)/39 = 1/39$, and it is larger for $n > 13$. So $c_1 = 1/39$ works for $n \geq 13$.

Hence $c_1 = 1/39$, $c_2 = 1/3$, $n_0 = 13$, and $(1/3)n^2 - 4n = \Theta(n^2)$. (Check $n = 13$: $f = 169/3 - 52 = 13/3$ and $c_1n^2 = 169/39 = 13/3$. Equal, as expected.)

EXAMPLE (FROM THE UNIT)

Show that $6n^3 \neq \Theta(n^2)$.

Suppose c_2 and n_0 exist with $6n^3 \leq c_2n^2$ for all $n \geq n_0$. Dividing by n^2 gives $n \leq c_2/6$ for all large n , which is impossible since n grows without limit. So no upper constant exists and $6n^3 \neq \Theta(n^2)$.

EXAMPLE

Show $2n + 3 = \Theta(n)$.

Lower: $2n \leq 2n + 3$ for all $n \geq 1$, so $c_1 = 2$. Upper: $2n + 3 \leq 3n \Leftrightarrow n \geq 3$, so $c_2 = 3$. Take $n_0 = 3$.

Exam tip: In the unit's working, both inequalities are called "right hand"; the second one ($c_1 \leq 1/3 - 4/n$) is the left-hand inequality. Also the unit writes " $n_0 \geq 13$ "; the constant is simply $n_0 = 13$ (any larger value also works).

Big O notation (upper bound)

Big O: $O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0 \}$; it gives an upper bound on $f(n)$ within a constant factor.

Big O: എല്ലാ $n \geq n_0$ -നും $0 \leq f(n) \leq c \cdot g(n)$ ആകുന്ന positive constants c, n_0 ഉള്ള functions $f(n)$ -ന്റെ set ആണ് $O(g(n))$; ഇത് $f(n)$ -ന് ഒരു constant factor വരെയുള്ള upper bound തരുന്നു.

$f(n) = O(g(n))$ means: from n_0 onwards, f never rises above $c \cdot g(n)$. It says "grows no faster than". Because it is only an upper bound, a function can be O of many things: $n + c$ is $O(n)$, and also $O(n^2)$, but $O(n)$ is the tight one and $O(n^2)$ is not tight. Θ is therefore stronger than O . Big O is mostly used to state worst-case running time.

മലയാളത്തിൽ

Big O ഒരു "ceiling" ആണ്: n_0 -ന് ശേഷം $f(n)$ ഒരിക്കലും $c \cdot g(n)$ -ന് മുകളിൽ പോകില്ല. KSRTC ബസ്സിന്റെ speed limit 80 ആണെങ്കിൽ, അത് 60-ൽ പോയാലും "limit 80" എന്ന statement ശരിയാണ്; അതുപോലെ n , $O(n^2)$ ആണ് എന്നതും ശരി, പക്ഷേ tight അല്ല. അതുകൊണ്ട് exam-ൽ എപ്പോഴും ഏറ്റവും ചെറിയ (tight) function എഴുതുക.

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, n_0 > 0 : 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$$

If $f(n) = \Theta(g(n))$ then $f(n) = O(g(n))$ (Θ implies O)

EXAMPLE (FROM THE UNIT)

Show $n^2 + 3n + 4 = O(n^2)$.

We want $n^2 + 3n + 4 \leq 2n^2$, i.e. $3n + 4 \leq n^2$. At $n = 4$: $16 \leq 16$ true (equality); at $n = 5$: $19 \leq 25$. For $n \geq 4$ it keeps holding because n^2 grows faster. So $c = 2, n_0 = 4$ (the unit's $n > 10$ also works, just a looser choice).

EXAMPLE (CHECK YOUR PROGRESS 2, Q4)

Prove that if $f(n) = n^2 + 2n + 5$ and $g(n) = n^2$, then $f(n) = O(g(n))$.

Method 1: for $n \geq 1$, $2n \leq 2n^2$ and $5 \leq 5n^2$. So $f(n) \leq n^2 + 2n^2 + 5n^2 = 8n^2$. Take $c = 8, n_0 = 1$.

Method 2: $f(n) \leq 2n^2 \Leftrightarrow 2n + 5 \leq n^2$. $n = 3$: $11 \leq 9$ false; $n = 4$: $13 \leq 16$ true. So $c = 2, n_0 = 4$.

Either way $f(n) = O(n^2)$.

EXAMPLE

Is $3n + 4 = O(n^2)$? Is it $O(n)$?

Both are true: $3n + 4 \leq n^2$ for $n \geq 4$ and $3n + 4 \leq 4n$ for $n \geq 4$. By convention we report the tighter bound, $O(n)$.

Ω -notation (lower bound)

Ω -notation: $\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0 \}$; it gives a lower bound on $f(n)$ within a constant factor.

Ω -notation: എല്ലാ $n \geq n_0$ -നും $0 \leq c \cdot g(n) \leq f(n)$ ആകുന്ന positive constants c, n_0 ഉള്ള functions $f(n)$ -ന്റെ set ആണ് $\Omega(g(n))$; ഇത് $f(n)$ -ന് ഒരു constant factor വരെയുള്ള lower bound തരുന്നു.

$f(n) = \Omega(g(n))$ means f grows at least as fast as g : from n_0 on, f stays on or above $c \cdot g(n)$. The unit uses Ω to describe best-case running time. Putting O and Ω together gives Θ : $f(n) = \Theta(g(n))$ exactly when $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

മലയാളത്തിൽ

Ω ഒരു "floor" ആണ്: n_0 -ന് ശേഷം $f(n) \geq c \cdot g(n)$ -ന് താഴെ പോകില്ല. "കുറഞ്ഞത് ഇത്രയെങ്കിലും സമയം എടുക്കും" എന്ന് പറയുന്നത് Ω ആണ്. Ceiling (O)-ഉം floor (Ω)-ഉം ഒരേ function $g(n)$ ആണെങ്കിൽ, $f(n) = \Theta(g(n))$.

$$f(n) = \Omega(g(n)) \Leftrightarrow \exists c > 0, n_0 : 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$$

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

EXAMPLE

Show $4n + 6 = \Theta(n)$ using O and Ω (the unit's example).

O : $4n + 6 \leq 5n \Leftrightarrow n \geq 6$, so $c = 5, n_0 = 6$. Ω : $1 \cdot n \leq 4n + 6$ for all $n \geq 1$, so $c = 1$. Both hold, so $4n + 6 = \Theta(n)$.

EXAMPLE

Is $n^2 = \Omega(n)$? Is $n = \Omega(n^2)$?

$n^2 \geq 1 \cdot n$ for all $n \geq 1$, so $n^2 = \Omega(n)$. But $n \geq c \cdot n^2$ means $n \leq 1/c$, which fails for large n , so $n \neq \Omega(n^2)$.

Exam tip: IGNOU says O is "used for worst case" and Ω "for best case". That is the usual use, but strictly O , Ω and Θ are bounds on functions and can be applied to any case, e.g. "insertion sort's worst case is $\Theta(n^2)$ ".

Simplifying with asymptotic notation and rules of big O

Rule of dominant term: in big O notation, constant factors and lower-order terms are dropped, so a polynomial of degree k is $O(n^k)$.

Rule of dominant term: big O notation-ൽ constant factors-ഉം lower-order terms-ഉം ഒഴിവാക്കുന്നു; അതിനാൽ degree k ഉള്ള polynomial $O(n^k)$ ആണ്.

An O -term inside an expression stands for "some function of that order". We can absorb smaller terms step by step until only the highest-order term is left. The six rules below let you simplify almost any growth function quickly.

മലയാളത്തിൽ

ഒരു വലിയ തുകയിൽ ചിലവ് പെസ നോക്കാറില്ലല്ലോ; ₹10,00,005-ൽ ആ 5 രൂപ ആരും ശ്രദ്ധിക്കില്ല. അതുപോലെ n വലുതാകുമ്പോൾ ഏറ്റവും വേഗം വളരുന്ന term മാത്രം പ്രധാനം, ബാക്കി terms-ഉം constants-ഉം കളയാം.

1. $f(n) = O(f(n))$ (every function bounds itself)
2. $a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 = O(n^k)$
3. $\log_a n = O(\log_b n)$ for any bases $a, b > 1$, so we just write $O(\log n)$
4. $\log_b n = O(n^c)$ for any $c > 0$ (logs are below every polynomial)
5. $n^k = O(b^n)$ for any $b > 1$ (polynomials are below exponentials)
6. $a^n = O(n!)$ (exponentials are below factorial)

EXAMPLE (FROM THE UNIT)

Simplify $f(n) = 3n^3 + 2n^2 + 4n + 3$.

$= 3n^3 + 2n^2 + O(n)$, since $4n + 3$ is $O(n)$
 $= 3n^3 + O(n^2)$, since $2n^2 + O(n)$ is $O(n^2)$
 $= O(n^3)$.

EXAMPLE (CHECK YOUR PROGRESS 1, Q5)

Give simplified big O for: $30n^2$, $10n^3 + 6n^2$, $5n \log n + 30n$, $\log n + 3n$, $\log n + 32$.

Function	Dominant term	Big O
$30n^2$	$30n^2$	$O(n^2)$
$10n^3 + 6n^2$	$10n^3$	$O(n^3)$
$5n \log n + 30n$	$5n \log n$	$O(n \log n)$
$\log n + 3n$	$3n$ (rule 4: $\log n = O(n)$)	$O(n)$
$\log n + 32$	$\log n$	$O(\log n)$

EXAMPLE

Why is $\log_2 n = O(\log_{10} n)$?

By change of base, $\log_2 n = \log_{10} n / \log_{10} 2 \approx 3.32 \cdot \log_{10} n$. It is a constant multiple, so $c = 3.33$ works. The base only changes the constant.

Exam tip: The unit's answer key gives $O(\log n)$ for " $\log n + 3n$ ". That is wrong: $3n$ dominates $\log n$, so the correct answer is $O(n)$.

Process of analysis: performance vs complexity, the RAM model

Time complexity: the amount of time (number of basic steps) an algorithm needs, as a function of input size n ; formally, the maximum time a Turing machine takes on any input of length n . **Space complexity:** the amount of memory an algorithm needs, as a function of input size n .

Time complexity: input size n -ന്റെ function ആയി ഒരു algorithm-ന് വേണ്ട സമയം (basic steps-ന്റെ എണ്ണം); formal ആയി, length n ഉള്ള ഏത് input-ലും ഒരു Turing machine എടുക്കുന്ന maximum time. **Space complexity:** input size n -ന്റെ function ആയി ഒരു algorithm-ന് വേണ്ട memory.

We analyse an algorithm to find its efficiency in using resources: CPU (time), memory (space), disk (I/O) and network (bandwidth). **Performance** is how much of a resource is actually used in one run; it depends on machine, compiler and so on. **Complexity** is how those requirements scale as the input grows. For machine-independent counting we use the **RAM (Random Access Machine) model**: every simple operation and every memory access costs one step, while loops and function calls cost according to how many times they run and what they contain.

മലയാളത്തിൽ

Performance എന്നത് ഒരു പ്രത്യേക computer-ൽ ഒരു run-ൽ കിട്ടുന്ന യഥാർത്ഥ സമയം; complexity എന്നത് n കൂടുമ്പോൾ അത് എങ്ങനെ വളരുന്നു എന്നത്. RAM model-ൽ $+$, $-$, $=$, if , call പോലുള്ള ഓരോ simple operation-നും ഒരു step, ഓരോ memory access-നും ഒരു step. Loop ആണെങ്കിൽ അത് എത്ര തവണ ഓടുന്നു എന്ന് നോക്കി കൂട്ടണം.

RAM model: simple operation ($+$, $-$, $=$, if , call) = 1 step; memory access = 1 step

Loops and subroutine calls: cost depends on data size and body

$O(1)$ constant: time does not depend on n

$O(n)$ linear: n doubles $\Rightarrow$ time doubles

$O(n^2)$ quadratic: n doubles $\Rightarrow$ time becomes 4 times

EXAMPLE

An algorithm takes 2 seconds for $n = 1000$. Estimate the time for $n = 2000$ if it is $O(1)$, $O(n)$, $O(n^2)$.

$O(1)$: still about 2 s. $O(n)$: $2 \times 2 = 4$ s. $O(n^2)$: $(2000/1000)^2 \times 2 = 4 \times 2 = 8$ s.

EXAMPLE

Space $O(n^2)$: what happens to memory when n doubles?

$(2n)^2 = 4n^2$, so the working storage becomes four times, exactly as the unit says.

Exam tip: RAM here stands for Random Access Machine (the unit writes "random access model"). It is an idealised machine based on the von Neumann computer; do not confuse it with RAM memory.

Analysing statements: sequence, if-else, loops, nested loops

Sequence rule: the time of a sequence of statements is the sum of their times. **Loop rule:** the time of a loop is the number of iterations multiplied by the time of its body.

Sequence rule: ഒരു കൂട്ടം statements-ന്റെ time അവയുടെ times-ന്റെ തുകയാണ്. **Loop rule:** ഒരു loop-ന്റെ time = iterations-ന്റെ എണ്ണം $\times$ body-യുടെ time.

Build the running time from the program structure. A sequence of simple statements is $O(1)$. For if-else, the worst case is the slower branch. A loop running n times with an $O(1)$ body is $O(n)$; the number of statements

inside only changes the constant. For nested loops, multiply: outer n times, inner m times gives $O(n \cdot m)$, and $O(n^2)$ when both run n times. A loop whose counter doubles runs about $\log_2 n$ times.

മലയാളത്തിൽ

ഒന്നിന് പിറകെ ഒന്നായി വരുന്ന statements-ന്റെ സമയം കൂട്ടുക; if-else-ൽ കൂടുതൽ സമയം എടുക്കുന്ന branch എടുക്കുക. Loop ആണെങ്കിൽ ഗുണിക്കുക. ഒരു ക്ലാസിൽ n കുട്ടികൾ, ഓരോരുത്തരും m പുസ്തകം വായിക്കണം എങ്കിൽ ആകെ $n \times m$ പുസ്തകം, അതാണ് nested loop. i ഓരോ തവണയും ഇരട്ടിയാകുന്ന loop ഏകദേശം $\log_2 n$ തവണ മാത്രമേ ഓടൂ.

Sequence: $T = T_1 + T_2 + \dots + T_k$; all simple $\Rightarrow O(1)$

if-else: worst case = $\max(T_{\text{then}}, T_{\text{else}})$

for ($i = 0$; $i < n$; $i++$) body $O(1) \Rightarrow O(n)$

nested $n \times m \Rightarrow O(n \cdot m)$; $n \times n \Rightarrow O(n^2)$

for ($i = 1$; $i \leq n$; $i *= 2$) $\Rightarrow \lfloor \log_2 n \rfloor + 1$ iterations $\Rightarrow O(\log n)$

EXAMPLE (FROM THE UNIT)

if (cond) { sequence 1 } else { sequence 2 }, where sequence 1 is $O(n^2)$ and sequence 2 is $O(1)$. Worst case?

Only one branch runs, but in the worst case it is the slower one, so the whole statement is $O(n^2)$.

EXAMPLE (CHECK YOUR PROGRESS 2, Q2)

Find the complexity of printMultiplicationTable. The unit's version uses C++ cout; here it is in C:

```
#include <stdio.h>

void printMultiplicationTable(int max)
{
    int i, j;
    for (i = 1; i <= max; i++) {
        for (j = 1; j <= max; j++)
            printf("%d ", i * j);
        printf("\n");
    }
}

int main(void)
{
    printMultiplicationTable(3);
    return 0;
}
```

```
1 2 3
2 4 6
3 6 9
```

The inner printf runs max times for each of the max outer iterations: $\max^2$ times. The newline printf runs max times. Total $\approx \max^2 + \max = O(\max^2)$, i.e. $O(n^2)$ with $n = \max$.

EXAMPLE (CHECK YOUR PROGRESS 2, Q3 AND Q5)

How many times does for (i = 1; i <= n; i *= 2) k = k + 1; run?

i takes the values 1, 2, 4, ..., 2^t while $2^t \leq n$, so $t = 0, 1, \dots, \lfloor \log_2 n \rfloor$: that is $\lfloor \log_2 n \rfloor + 1$ iterations, which is $O(\log n)$. A test program printed:

```
n = 1  loop runs 1 times
n = 2  loop runs 2 times
n = 3  loop runs 2 times
n = 4  loop runs 3 times
n = 7  loop runs 3 times
n = 8  loop runs 4 times
n = 16 loop runs 5 times
n = 20 loop runs 5 times
```

For $n = 8$: $\lfloor \log_2 8 \rfloor + 1 = 3 + 1 = 4$. For $n = 20$: $\lfloor 4.32 \rfloor + 1 = 5$. Both match.

Exam tip: Check Your Progress 2 Q5 is printed as `for (i=1; i<= n; i*2)`. In C, `i*2` computes a value and throws it away (gcc warns "statement with no effect"), so `i` stays 1 and the loop never ends for $n \geq 1$. The intended loop is `i *= 2`, which runs $\lfloor \log_2 n \rfloor + 1$ times, i.e. $O(\log n)$; the key's answer "log n" is the order, not the exact count. Also, in Q2 the `cout << endl` is not inside the inner loop (no braces), so the key's " $\max^*(2*\max)$ " count is off; the result $O(n^2)$ is still right.

Counting steps: the partial-sum function

Step count: the total number of times the basic statements of an algorithm execute, expressed as a function of n ; its dominant term gives the order of the running time.

Step count: ഒരു algorithm-ലെ basic statements ആകെ എത്ര തവണ execute ചെയ്യുന്നു എന്നത് n -ന്റെ function ആയി; അതിലെ dominant term running time-ന്റെ order തരുന്നു.

To count steps, count every execution of every line, including each test of the loop condition. A for loop `for (i = 1; i <= n; i++)` costs 1 initialisation, $n + 1$ tests (the last test fails) and n increments. Then apply O , Ω and Θ to the total.

മലയാളത്തിൽ

ഓരോ line-ഉം എത്ര തവണ ഓടുന്നു എന്ന് ഒരു table-ൽ എഴുതി കൂട്ടിയാൽ മതി.

ശ്രദ്ധിക്കേണ്ടത്: loop condition $n + 1$ തവണ check ചെയ്യും, കാരണം അവസാനത്തെ ഒരു check false ആയിട്ടാണ് loop നിൽക്കുന്നത്, ബസ് stop-ൽ ഇറങ്ങാൻ ഒരു stop കൂടി നോക്കുന്നതു പോലെ.

```

#include <stdio.h>

int psum(int n)
{
    int i, partial_sum;

    partial_sum = 0;                /* Line 1 */
    for (i = 1; i <= n; i++) {      /* Line 2 */
        partial_sum = partial_sum + i * i; /* Line 3 */
    }
    return partial_sum;            /* Line 4 */
}

int main(void)
{
    printf("psum(3) = %d\n", psum(3));
    printf("psum(10) = %d\n", psum(10));
    return 0;
}

```

```

psum(3) = 14
psum(10) = 385

```

EXAMPLE (FROM THE UNIT, RECOUNTED)

Count the steps of $\text{psum}(n) = 1^2 + 2^2 + \dots + n^2$.

Line	Statement	Times executed
1	partial_sum = 0	1
2	i = 1	1
2	i <= n	n + 1
2	i++	n
3	partial_sum = partial_sum + i*i	n
4	return partial_sum	1
Total		3n + 4

Line 2 alone is $1 + (n + 1) + n = 2n + 2$ steps. Adding lines 1, 3 and 4 gives $T(n) = 3n + 4$. Check for $n = 3$: $1 + 1 + 4 + 3 + 3 + 1 = 13 = 3 \cdot 3 + 4$.

O: $3n + 4 \leq 4n$ for $n \geq 4$ ($c = 4$). Ω : $3n + 4 \geq 1 \cdot n$ for $n \geq 1$ ($c = 1$). So $T(n) = \Theta(n)$, hence also $O(n)$ and $\Omega(n)$. It is also $O(n \log n)$ and $O(n^2)$, but $O(n)$ is the tight answer.

EXAMPLE

Count steps for `for (i = 0; i < n; i++) for (j = 0; j < n; j++) x++;`

Outer: $1 + (n + 1) + n = 2n + 2$. Inner loop, per outer iteration: $1 + (n + 1) + n = 2n + 2$, done n times: $2n^2 + 2n$. Body `x++`: n^2 . Total = $3n^2 + 4n + 2 = \Theta(n^2)$.

Exam tip: The unit writes the total as " $1 + (n+1) + n + 1 = 2n + 3$ ". That leaves out line 3, which runs n times, and line 4. The full count is $3n + 4$. The conclusion $\Theta(n)$ is unchanged. Also, " $c = 3$ gives $3n > 2n + 3$ " only holds for $n > 3$, so always state n_0 .

Exact analysis of insertion sort; best, worst and average case

Worst case: the maximum running time over all inputs of size n (an upper bound for any input). **Best case:** the minimum running time over all inputs of size n . **Average case:** the average running time over all inputs of size n .

Worst case: size n ഉള്ള എല്ലാ inputs-ലും വെച്ച് ഏറ്റവും കൂടിയ running time (ഏത് input-നും upper bound). **Best case:** size n ഉള്ള എല്ലാ inputs-ലും വെച്ച് ഏറ്റവും കുറഞ്ഞ running time. **Average case:** size n ഉള്ള എല്ലാ inputs-ന്റെയും ശരാശരി running time.

Insertion sort takes each element $A[j]$ (the key) and shifts the larger elements of the sorted part $A[1..j-1]$ one place right, then drops the key into the gap. Let t_j be the number of times the while-condition is tested for that j . Multiply each line's cost by how often it runs and add. The input order decides t_j : already sorted gives the best case, reverse sorted the worst case, random order the average case.

മലയാളത്തിൽ

കയ്യിലെ ചീട്ടുകൾ (cards) അടുക്കുന്നതു പോലെയാണ് insertion sort: ഓരോ പുതിയ card-ഉം എടുത്ത്, വലുതായവയെ ഒന്ന് വലത്തോട്ട് നീക്കി, ശരിയായ സ്ഥാനത്ത് തിരുക്കുന്നു. Cards നേരത്തെ അടുക്കിയതാണെങ്കിൽ ഒന്നും നീക്കേണ്ട (best case, n). തലതിരിച്ചാണെങ്കിൽ ഓരോന്നും മുഴുവൻ നീക്കണം (worst case, n^2).

Line	Pseudocode (1-based)	Cost	Times
1	for j = 2 to length[A] do	c_1	n
2	key = A[j]	c_2	n - 1
3	i = j - 1	c_3	n - 1
4	while (i > 0) and (A[i] > key) do	c_4	$\sum_{j=2}^n t_j$
5	A[i+1] = A[i]	c_5	$\sum_{j=2}^n (t_j - 1)$
6	i = i - 1	c_6	$\sum_{j=2}^n (t_j - 1)$
7	A[i+1] = key	c_7	n - 1

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum t_j + c_5 \sum (t_j - 1) + c_6 \sum (t_j - 1) + c_7(n-1)$$

$$\text{Best (sorted): } t_j = 1 \Rightarrow T(n) = c_1 n + (c_2 + c_3 + c_4 + c_7)(n-1) = \Theta(n)$$

$$\text{Worst (reverse sorted): } t_j = j \Rightarrow \sum t_j = n(n+1)/2 - 1, \sum (t_j - 1) = n(n-1)/2 \Rightarrow T(n) = \Theta(n^2)$$

$$\text{Average (random): } t_j \approx j/2 \Rightarrow T(n) = \Theta(n^2)$$

$$T_{\text{best}}(n) \leq T_{\text{avg}}(n) \leq T_{\text{worst}}(n)$$

The same algorithm in C (0-based array, so the loop starts at j = 1 and the while test is i >= 0):

```

#include <stdio.h>

void insertion_sort(int A[], int n)
{
    int i, j, key;
    for (j = 1; j < n; j++) {          /* for j = 2 to n (1-based) */
        key = A[j];
        i = j - 1;
        while (i >= 0 && A[i] > key) { /* shift bigger elements right */
            A[i + 1] = A[i];
            i = i - 1;
        }
        A[i + 1] = key;
    }
}

int main(void)
{
    int A[] = {5, 2, 4, 6, 1, 3};
    int n = 6, k;
    insertion_sort(A, n);
    for (k = 0; k < n; k++)
        printf("%d ", A[k]);
    printf("\n");
    return 0;
}

```

1 2 3 4 5 6

EXAMPLE (TRACE)

Trace insertion sort on $A = [5, 2, 4, 6, 1, 3]$ (j in 1-based numbering).

j	key	Array after inserting key	Shifts	t_j
start	–	5 2 4 6 1 3	–	–
2	2	2 5 4 6 1 3	1	2
3	4	2 4 5 6 1 3	1	2
4	6	2 4 5 6 1 3	0	1
5	1	1 2 4 5 6 3	4	5
6	3	1 2 3 4 5 6	3	4

$\sum t_j = 2 + 2 + 1 + 5 + 4 = 14$ while-tests and $\sum (t_j - 1) = 9$ shifts (confirmed by an instrumented run).

EXAMPLE (BEST VS WORST, $N = 5$)

Sorted input [1, 2, 3, 4, 5]: $t_j = 1$ for $j = 2..5$, so line 4 runs $4 = n - 1$ times and no shifts happen.

Reverse input [5, 4, 3, 2, 1]: $t_j = j$, so line 4 runs $2 + 3 + 4 + 5 = 14 = 5 \cdot 6 / 2 - 1$ times, and lines 5 and 6 run $1 + 2 + 3 + 4 = 10 = 5 \cdot 4 / 2$ times each. An instrumented run gave exactly 4/0 and 14/10.

Exam tip: The unit's cost table repeats c_4 and shifts later costs, then its formula uses $c_1 \dots c_7$; use one cost per line as above. Its best-case $T(n)$ also drops the $c_7(n - 1)$ term for line 7, which still runs $n - 1$ times; the answer $\Theta(n)$ is unchanged. The average case is not just "somewhere in between": on random input about half the sorted part is shifted, so it is $\Theta(n^2)$, like the worst case.

Calculation of storage (space) complexity

Space complexity: the amount of memory a program needs for its proper execution, as a function of input size. For a Turing machine, it is the (worst case) maximum length of tape used to process an input of length n .

Space complexity: ഒരു program ശരിയായി execute ചെയ്യാൻ വേണ്ട memory, input size-ന്റെ function ആയി. Turing machine-ന്, length n ഉള്ള input process ചെയ്യാൻ വേണ്ട tape-ന്റെ (worst case) maximum length.

A slow program still finishes, but a program that needs more memory than the machine has cannot run at all; that is why the unit calls space more critical than time. The good news is that memory can be reused during execution, unlike time. For an iterative program, count variables and allocated arrays. For a recursive program, the space at any moment is the sum over all calls active at that moment (their parameters, locals and return addresses); n nested calls need $O(n)$ space.

മലയാളത്തിൽ

സമയം കൂടുതലെടുത്താൽ കാത്തിരുന്നാൽ മതി; പക്ഷേ memory തികഞ്ഞില്ലെങ്കിൽ program ഓടുകയേ ഇല്ല. Iterative program-ൽ variables-ഉം arrays-ഉം എണ്ണിയാൽ മതി. Recursive program-ൽ ഒരേ സമയം stack-ൽ എത്ര calls "തുറന്നിരിക്കുന്നു" എന്ന് നോക്കണം; ഒരു പാത്രത്തിന് മുകളിൽ മറ്റൊന്ന് അടുക്കി വെക്കുന്നത് പോലെ, ഏറ്റവും ഉയരമുള്ള അടുക്കാണ് space.

Iterative: space = variables + arrays (e.g. k ints $\Rightarrow O(1)$; array of n $\Rightarrow O(n)$)

Recursive: space = (max recursion depth) $\times$ (space per call)

PSPACE: decision problems solvable by a Turing machine using polynomial memory and unlimited time

The unit's binary recursion example ($f(0) = f(1) = 1$, $f(n) = f(n-1) + f(n-2)$) in C:

```
#include <stdio.h>

int f(int n)
{
    if (n == 0 || n == 1)
        return 1;
    return f(n - 1) + f(n - 2);
}

int main(void)
{
    int n;
    for (n = 0; n <= 6; n++)
        printf("f(%d) = %d\n", n, f(n));
    return 0;
}
```

```
f(0) = 1
f(1) = 1
f(2) = 2
f(3) = 3
f(4) = 5
f(5) = 8
f(6) = 13
```

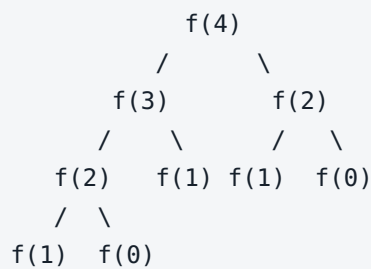
EXAMPLE (BINARY RECURSION, FROM THE UNIT)

Find the time and space complexity of $f(n)$.

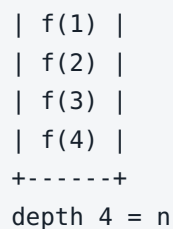
Time: number of calls $C(n) = C(n-1) + C(n-2) + 1$ with $C(0) = C(1) = 1$, giving 1, 1, 3, 5, 9, 15, 25, ... This grows like $f(n)$ itself, about 1.618^n , so time is exponential, $O(2^n)$.

Space: the calls are not all alive at once. The deepest chain is $f(n) \rightarrow f(n-1) \rightarrow \dots \rightarrow f(1)$, which is n frames. When $f(n-1)$ returns, its frames are freed and reused by $f(n-2)$. So space = $O(n)$.

Call tree for $f(4)$



Stack at the deepest point



9 calls in total

EXAMPLE

Space complexity of a function that reverses an array of n ints in place using one temporary variable?

Besides the input array, it uses only an index or two and one temp: $O(1)$ extra space.

Exam tip: The unit states "Space Complexity: $O(\exp n)$ " for binary recursion. That is wrong: time is exponential, but space is only $O(n)$, because at most n calls are on the stack at any moment and space is reused. This is exactly the "space can be reused" point the unit makes itself.

Worked example: GCD by repeated subtraction

GCD (greatest common divisor): the largest positive integer that divides both m and n . Subtraction method: while $m > 0$, swap so that $m \geq n$, then replace m by $m - n$; when m reaches 0, n is the GCD.

GCD (greatest common divisor): m -നെയും n -നെയും ഒരുപോലെ ശിഷ്ടമില്ലാതെ ഹരിക്കുന്ന ഏറ്റവും വലിയ positive integer. Subtraction method: $m > 0$ ആയിരിക്കുമ്പോൾ $m \geq n$ ആകാൻ swap ചെയ്ത് m -നെ $m - n$ ആക്കുക; $m, 0$ ആകുമ്പോൾ n ആണ് GCD.

This is the unit's example for both space and time. It uses only three integers m , n and t , so its space is $O(1)$. Its time depends on the number of loop iterations, which depends on both m and n . The unit's C code is correct; below it is with a test driver.

മലയാളത്തിൽ

വലിയ സംഖ്യയിൽ നിന്ന് ചെറുതിനെ വീണ്ടും വീണ്ടും കുറച്ചുകൊണ്ടിരിക്കുന്നു, m പൂജ്യമാകുന്നത് വരെ. Memory-ക്ക് മൂന്ന് `int` മാത്രം (m, n, t), അതിനാൽ space $O(1)$. പക്ഷേ സമയം m, n -ന്റെ values അനുസരിച്ച് മാറും: $m = n$ ആണെങ്കിൽ ഒറ്റ round, $n = 1$ ആണെങ്കിൽ m rounds.

```
#include <stdio.h>

int gcd(int m, int n)
/* Precondition: m > 0 and n > 0 */
{
    while (m > 0) {
        if (n > m) {
            int t = m; m = n; n = t;    /* swap m and n */
        }
        /* now m >= n > 0 */
        m -= n;
    }
    return n;
}

int main(void)
{
    printf("gcd(12, 18) = %d\n", gcd(12, 18));
    printf("gcd(7, 7)   = %d\n", gcd(7, 7));
    printf("gcd(5, 1)   = %d\n", gcd(5, 1));
    return 0;
}
```

```
gcd(12, 18) = 6
gcd(7, 7)   = 7
gcd(5, 1)   = 1
```

EXAMPLE (TRACE)

Trace `gcd(12, 18)`.

Iteration	m, n at start	After swap	After m -= n
1	12, 18	18, 12	6, 12
2	6, 12	12, 6	6, 6
3	6, 6	6, 6	0, 6

$m = 0$, loop ends, return $n = 6$.

EXAMPLE (BEST AND WORST CASE)

Best case: $m = n$. One subtraction makes $m = 0$: 1 iteration, $O(1)$.

Worst case: $n = 1$ (or $m = 1$). Each iteration subtracts only 1, so $\text{gcd}(m, 1)$ takes m iterations; $\text{gcd}(5, 1)$: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$, 5 iterations. In general the time is $O(\max(m, n))$.

Exam tip: Check Your Progress 3 calls this "Euclid's algorithm"; its space complexity is $O(1)$ (the key is right). The unit writes the worst case as " $O(n)$ "; say $O(\max(m, n))$ to be clear which number is meant. The faster modulo version ($r = m \% n$) of Euclid's algorithm needs only $O(\log \min(m, n))$ iterations.

Calculation of time complexity: constant code, binary search, TSP

Binary search: a search in a sorted list that compares the key with the middle element and halves the search interval each time; it takes $O(\log n)$ time.

Binary search: sorted list-ൽ key-യെ middle element-നോട് compare ചെയ്ത് ഓരോ തവണയും search interval പകുതിയാക്കുന്ന search; ഇതിന് $O(\log n)$ time വേണം.

Straight-line code on scalar variables, such as $x = 4*y + 3$; $z = z + 1$; $p = 1$;, takes constant time whatever the values: each line is $O(1)$. Binary search shrinks the interval $N, N/2, N/4, \dots, 1$; taking $\log_2$ gives $\log_2 N, \log_2 N - 1, \dots, 0$, which has $\log_2 N + 1$ terms, so $O(\log N)$. The travelling salesman problem (TSP) by brute force must look at $n!$ orderings, which is at least exponential.

മലയാളത്തിൽ

Binary search നിഘണ്ടു (dictionary) നോക്കുന്നതു പോലെയാണ്: നടുവിൽ തുറക്കുക, വാക്ക് മുന്നിലാണോ പിന്നിലാണോ എന്ന് നോക്കി പകുതി കളയുക. 1024

പേജുണ്ടെങ്കിലും ഏകദേശം 11 തവണ തുറന്നാൽ മതി. TSP-ൽ എല്ലാ വഴികളും ($n!$) try ചെയ്യേണ്ടി വരും; 20 നഗരങ്ങൾക്ക് പോലും അത് കോടിക്കണക്കിന് കോടി വഴികളാണ്.

Interval sizes: $N, N/2, N/4, \dots, 2, 1 \Rightarrow \log_2 N + 1$ iterations (N a power of 2)

In general, at most $\lceil \log_2 N \rceil + 1$ iterations $\Rightarrow O(\log N)$

TSP brute force: $n!$ orderings; $n! \geq 2^{n-1} \Rightarrow n! = \Omega(2^n)$

```
#include <stdio.h>

int binary_search(int A[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (A[mid] == key)
            return mid;
        else if (A[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void)
{
    int A[] = {3, 8, 12, 17, 25, 31, 40, 52};
    printf("index of 31 = %d\n", binary_search(A, 8, 31));
    printf("index of 10 = %d\n", binary_search(A, 8, 10));
    return 0;
}
```

```
index of 31 = 5
index of 10 = -1
```

EXAMPLE (TRACE)

Search for 31 in A = [3, 8, 12, 17, 25, 31, 40, 52] (indices 0–7).

Iteration	low	high	mid	A[mid]	Action
1	0	7	3	17	17 < 31, low = 4
2	4	7	5	31	found, return 5

Searching for 10 (absent): mid = 3 (17 > 10, high = 2), mid = 1 (8 < 10, low = 2), mid = 2 (12 > 10, high = 1), low > high, return -1. That is 3 iterations; the worst case for n = 8 is $\lfloor \log_2 8 \rfloor + 1 = 4$.

EXAMPLE

At most how many iterations does binary search need for 1,000,000 elements?

$2^{19} = 524,288 \leq 10^6 < 2^{20} = 1,048,576$, so $\lfloor \log_2 10^6 \rfloor = 19$ and at most $19 + 1 = 20$ iterations. Linear search could need 1,000,000.

EXAMPLE (TSP)

Show $n! = \Omega(2^n)$.

$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$. Each of the $n - 1$ factors $2, 3, \dots, n$ is ≥ 2 , so $n! \geq 2^{n-1} = (1/2) \cdot 2^n$. With $c = 1/2$ and $n_0 = 1$, $n! = \Omega(2^n)$. For $n = 10$: $10! = 3,628,800$ while $2^9 = 512$.

Exam tip: The unit's " $n! > 2(n-1)$ " is an extraction of $n! \geq 2^{n-1}$ (a power, not a product). Also, $n!$ counts every ordering; if the start city is fixed and a tour and its reverse are the same, there are $(n - 1)!/2$ distinct tours, still exponential. No polynomial-time algorithm for TSP is known.

The complexity ladder and comparison tables

Complexity ladder: the ordering of common growth rates, $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$, from slowest-growing (best) to fastest-growing (worst).

Complexity ladder: സാധാരണ growth rates-ന്റെ ക്രമം, $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$, ഏറ്റവും പതുക്കെ വളരുന്നത് (best) മുതൽ ഏറ്റവും വേഗം വളരുന്നത് (worst) വരെ.

Constant time (array access $A[i]$) does not grow at all. Logarithmic (binary search) grows very slowly. Linear (one pass over an array) grows in step with n . $n \log n$ (merge sort) is the best possible worst case for comparison-based sorting. Polynomial n^k with $k > 2$ is rarely practical; selection sort is $O(n^2)$. Exponential 2^n is the most dangerous: useless except for tiny inputs.

മലയാളത്തിൽ

ഒരു കോണിയുടെ പടികൾ പോലെ: താഴത്തെ പടി $O(1)$ ഏറ്റവും നല്ലത്, മുകളിലേക്ക് പോകുന്നോം മോശം. Table നോക്കിയാൽ മനസ്സിലാകും: $n = 1000$ -ന് $\log n$ വെറും 10, n^2 പത്ത് ലക്ഷം, 2^n ആകട്ടെ 301 അക്കമുള്ള സംഖ്യ! അതുകൊണ്ടാണ് exponential algorithms ചെറിയ input-ന് മാത്രം ഉപയോഗിക്കുന്നത്.

Notation	Name	Example
$O(1)$	Constant	Accessing one array element $A[i]$
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Looping over an array of n elements
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Insertion sort (worst case), selection sort
$O(n^3)$	Cubic	Standard multiplication of two $n \times n$ matrices
$O(n^c)$	Polynomial	Any fixed power c
$O(c^n)$	Exponential	Binary recursion $f(n-1) + f(n-2)$
$O(n!)$	Factorial	Brute-force TSP (all orderings)

Array size N	$\log_2 N$	N	N^2	2^N
8	3	8	64	256
128	7	128	16,384	3.4×10^{38}
256	8	256	65,536	1.16×10^{77}
1000	≈ 10	1000	1 million	1.07×10^{301}
100,000	≈ 17	100,000	10 billion	too large

EXAMPLE

Arrange in increasing order of growth: n^2 , 2^n , $\log n$, $n \log n$, 1 , $n!$, n .

$1 < \log n < n < n \log n < n^2 < 2^n < n!$.

EXAMPLE

A computer does 10^9 steps per second. How long for $n = 10^6$ with an $O(n \log n)$ and an $O(n^2)$ algorithm?

$n \log_2 n \approx 10^6 \times 20 = 2 \times 10^7$ steps ≈ 0.02 s. $n^2 = 10^{12}$ steps ≈ 1000 s ≈ 17 minutes.

Exam tip: The unit's Table 1.1 lists "matrix multiplication" under $O(n^2)$. The standard triple-loop multiplication of two $n \times n$ matrices is $O(n^3)$ (n^2 entries, each a sum of n products); only matrix addition is $O(n^2)$. The unit's ordering " $O(1) < O(\log n) < O(n \log n) < \dots$ " also skips $O(n)$, which comes between $\log n$ and $n \log n$. 2^{256} is 1.158×10^{77} , so 1.16 rounds better than the printed 1.15.

Quick recap

- An algorithm is a finite sequence of definite, effective instructions that terminates with correct output; five features: input, output, definiteness, effectiveness, termination.
- P: decision problems solvable in polynomial time by a deterministic machine; NP: by a non-deterministic machine. PSPACE: polynomial memory.
- Complexity is the growth rate of time or space with input size; asymptotic analysis ignores machine and compiler.
- Θ : $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$ (tight bound).
- O : $f(n) \leq c g(n)$ for $n \geq n_0$ (upper bound). Ω : $f(n) \geq c g(n)$ for $n \geq n_0$ (lower bound). $\Theta = O$ and Ω together.
- Drop constants and lower-order terms; log base does not matter; $\log < \text{polynomial} < \text{exponential} < \text{factorial}$.
- RAM model: each simple operation and memory access is one step.
- Sequence: add. if-else: take the slower branch. Loop: iterations $\times$ body. Nested: multiply. Doubling counter: $\lfloor \log_2 n \rfloor + 1$ iterations.
- psum(n) executes $3n + 4$ steps, which is $\Theta(n)$.
- Insertion sort: best (sorted) $\Theta(n)$, worst (reverse) $\Theta(n^2)$, average $\Theta(n^2)$.
- Space: iterative = variables + arrays; recursive = maximum depth $\times$ space per call. Binary recursion: time exponential, space $O(n)$.
- Subtraction GCD: space $O(1)$; time 1 iteration ($m = n$) to $\max(m, n)$ iterations ($n = 1$).
- Binary search: $O(\log n)$; brute-force TSP: $n! = \Omega(2^n)$.
- Ladder: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$.

Check yourself

Q. 1. True or false: $9n + 12$ and $1000n + 400000$ are both $O(n)$.

True. $9n + 12 \leq 10n$ for $n \geq 12$, and $1000n + 400000 \leq 1001n$ for $n \geq 400000$. Constants do not affect the order.

Q. 2. True or false: if $f(n) = O(g(n))$ and $h(n) = O(g(n))$, then $f(n) + h(n) = O(g(n))$.

True. $f(n) \leq c_1 g(n)$ for $n \geq n_1$ and $h(n) \leq c_2 g(n)$ for $n \geq n_2$. So $f(n) + h(n) \leq (c_1 + c_2)g(n)$ for $n \geq \max(n_1, n_2)$.

Q. 3. True or false: if $f(n) = n^2 + 3n$ and $g(n) = 6000n + 34000$, then $O(f(n)) < O(g(n))$. Also: does asymptotic complexity depend on hardware?

False. $f(n) = O(n^2)$ and $g(n) = O(n)$; n^2 grows faster, so $O(g(n)) < O(f(n))$ even though g has large constants (f overtakes g once $n^2 + 3n > 6000n + 34000$, i.e. from $n = 6003$).

No. Asymptotic complexity is a property of the algorithm; hardware changes only the constant factors, which the notation ignores. So statement 4 of Check Your Progress 1 is False.

Q. 4. True or false: the set of algorithms whose order is $O(1)$ would run in the same time.

The unit's key says True, meaning each such algorithm takes time independent of n . Strictly the statement is False: $O(1)$ only says the time is bounded by some constant, and different $O(1)$ algorithms can have very different constants (one step vs a million steps). Write: "their running time does not grow with n , but it need not be the same".

Q. 5. Prove that $(1/2)n^2 - 3n = \Theta(n^2)$. Give c_1, c_2, n_0 .

Divide by n^2 : need $c_1 \leq 1/2 - 3/n \leq c_2$. The right side holds for all $n \geq 1$ with $c_2 = 1/2$. For the left side, at $n = 7$: $1/2 - 3/7 = 1/14$, and it increases with n . So $c_1 = 1/14, c_2 = 1/2, n_0 = 7$.

Q. 6. Find the exact number of times `x++` runs, and the order: `for (i = 1; i <= n; i++) for (j = 1; j <= i; j++) x++;`

For each i the inner loop runs i times. Total = $1 + 2 + \dots + n = n(n + 1)/2$. For $n = 4$: 10. Order $\Theta(n^2)$, even though the inner loop is "shorter" on average; this is the same sum that appears in insertion sort's worst case.

Q. 7. What is the running time of: `for (i = 1; i <= n; i *= 2) { j = 1; }` And how many times for $n = 100$?

$i = 1, 2, 4, 8, 16, 32, 64$ ($128 > 100$ stops): 7 iterations = $\lfloor \log_2 100 \rfloor + 1 = 6 + 1$. Running time $O(\log n)$.

Q. 8. Why is space complexity said to be more critical than time complexity? What is the space complexity of the subtraction (Euclid) GCD algorithm?

If an algorithm is slow, it still finishes, only later. If it needs more memory than the machine has, it cannot run at all. Some relief: memory can be reused during execution.

The GCD code uses only three integers m, n, t whatever the inputs, so its space complexity is $O(1)$.

Q. 9. For insertion sort on $[4, 3, 2, 1]$, count the while-tests (line 4) and the shifts (line 5). Which case is this?

It is reverse sorted, the worst case, so $t_j = j$. While-tests = $2 + 3 + 4 = 9 = 4 \cdot 5 / 2 - 1$. Shifts = $1 + 2 + 3 = 6 = 4 \cdot 3 / 2$. Running time $\Theta(n^2)$.

Q. 10. A recursive function $\text{sum}(n)$ returns $n + \text{sum}(n - 1)$ with $\text{sum}(0) = 0$. Give its time and space complexity.

There are $n + 1$ calls, each doing $O(1)$ work: time $O(n)$. The calls $\text{sum}(n), \text{sum}(n-1), \dots, \text{sum}(0)$ are all on the stack at once at the deepest point, so space $O(n)$. An iterative loop does the same in $O(1)$ space.

UNIT 2

Arrays

ഈ unit-ൽ ഏറ്റവും ലളിതമായ data structure ആയ array പഠിക്കുന്നു: declaration, memory, strings, multidimensional arrays, pointers-മായുള്ള ബന്ധം. തുടർന്ന് sparse matrix-ന്റെ 3-tuple രൂപം, polynomial addition, row major / column major representation, address calculation എന്നിവ C programs സഹിതം കാണുന്നു.

Arrays: declaration, size and bounds

Initializing arrays

Strings as character arrays

Multidimensional arrays

Arrays and pointers

Sparse matrices: triangular and tridiagonal

3-tuple representation of a sparse matrix

Polynomials using arrays

Representation of arrays in memory

Row major representation

Column major representation

Applications of arrays

Arrays: declaration, size and bounds

Array: a finite, ordered set of homogeneous elements (all of the same data type) stored in contiguous memory locations and accessed by an index.

Array: ഒരു data type-ലുള്ള elements-ന്റെ, contiguous memory locations-ൽ സൂക്ഷിച്ചിരിക്കുന്ന, index ഉപയോഗിച്ച് access ചെയ്യുന്ന, ഒരു finite ordered set.

A one-dimensional array holds all integers, or all characters, or all floats, but never a mix. In C it is declared as `data_type array_name[expression];` where the expression gives the number of elements. The smallest index is the **lower bound** (always 0 in C) and the largest is the **upper bound**. Total memory is the size of one element times the number of elements.

മലയാളത്തിൽ

ഒരു train-ലെ ഒരു തരം compartments ഒന്നിനു പിറകെ ഒന്നായി ചേർത്തു വെച്ചതു പോലെയാണ് array. എല്ലാ compartment-ഉം ഒരു size, ഒരു തരം (int മാത്രം, അല്ലെങ്കിൽ char മാത്രം). ഓരോന്നിനും ഒരു number (index) ഉണ്ട്; C-ൽ അത് 0-ൽ തുടങ്ങും. `int a[100];` എന്നാൽ `a[0]` മുതൽ `a[99]` വരെ 100 integers.

Declaration: `data_type array_name[size];`

Memory required (bytes) = `sizeof(data type) × number of elements`

Number of elements (range) = $UB - LB + 1$

In C: $LB = 0$, $UB = size - 1$

Address of `a[i]` = $Base + w \times (i - LB)$ (w = bytes per element)

EXAMPLE 1

How much memory does `int a[100];` need if an `int` takes 4 bytes? What are its bounds?

Memory = $4 \times 100 = 400$ bytes. $LB = 0$, $UB = 99$, number of elements = $99 - 0 + 1 = 100$.

EXAMPLE 2

An array `A[5 ... 20]` of 2-byte elements starts at address 500. Find the number of elements and the address of `A[12]`.

Number of elements = $20 - 5 + 1 = 16$.

Address of `A[12]` = $500 + 2 \times (12 - 5) = 500 + 14 = 514$.

EXAMPLE 3

`int a[10];` starts at 1000 (`int` = 4 bytes). Find the address of `a[7]`.

Address = $1000 + 4 \times (7 - 0) = 1028$.

Exam tip: The unit says the range (number of elements) is "upper bound – lower bound". That is off by one: the correct count is $UB - LB + 1$ (for `int a[100]`, $99 - 0 + 1 = 100$, not 99).

Initializing arrays

Array initialization: assigning values to array elements in the declaration itself, using a brace-enclosed list such as `int a[4] = {34, 60, 93, 2};`.

Array initialization: declaration-ൽ തന്നെ `int a[4] = {34, 60, 93, 2};` പോലെ braces-നുള്ളിലെ list ഉപയോഗിച്ച് array elements-ന് values നൽകുന്നത്.

Values can be stored during execution, or given at declaration time. If the array is initialized at declaration, the size may be left out and the compiler counts the values. Until elements are given values, a local array holds garbage.

മലയാളത്തിൽ

പുതിയ വീട്ടിലേക്ക് മാറുമ്പോൾ ഷെൽഫിൽ മുമ്പുള്ളവർ വെച്ച പഴയ സാധനങ്ങൾ കാണും; അതുപോലെ values കൊടുക്കാത്ത local array-ൽ garbage values ഉണ്ടാകും. Declaration-ൽ തന്നെ values കൊടുത്താൽ size എഴുതേണ്ട, compiler എണ്ണിക്കോളും.

```
int a[4] = {34, 60, 93, 2};  
int b[] = {2, 3, 4, 5}; (size taken as 4)  
float c[] = {-4, 6, 81, -60}; (size taken as 4)
```

EXAMPLE

What is the size of float c[] = {-4, 6, 81, -60}; if a float takes 4 bytes? What does int d[5] = {1, 2}; contain?

c has 4 elements, so $4 \times 4 = 16$ bytes.

d = {1, 2, 0, 0, 0}: when a partial list is given, C sets the remaining elements to 0.

Exam tip: Check Your Progress 1, Q1: "If the array is initialized at the time of declaration, then the dimension of the array is optional." Garbage values apply only to arrays with no initializer at all (global and static arrays start as 0).

Strings as character arrays

String: a one-dimensional array of characters terminated by the null character '\0' (whose value is zero).

String: null character '\0' (value പൂജ്യം) കൊണ്ട് അവസാനിക്കുന്ന ഒരു one-dimensional character array.

C does not check array bounds; checking them is the programmer's job. The most common array is the string. Each character takes one byte, stored contiguously, and the last one is always '\0'. With a string literal like "sentence\n", C adds the '\0' automatically. Note that '\0' (value 0) and '0' (the digit character, ASCII 48) are different.

മലയാളത്തിൽ

String എന്നത് characters നിറത്തി വെച്ച ഒരു array ആണ്, അവസാനം ഒരു '\0' "എൻഡ്സ്റ്റോപ്പ്" ഉണ്ടാകും. ഈ '\0' കണ്ടാലാണ് string എവിടെ തീരുന്നു എന്ന് printf പോലുള്ള functions അറിയുന്നത്. Double quotes-ൽ എഴുതുമ്പോൾ C തന്നെ '\0' ചേർക്കും; single characters ആയി എഴുതുമ്പോൾ നമ്മൾ തന്നെ '\0' ചേർക്കണം.

```
char message[] = {'e','x','a','m','p','l','e','\0'};
char name[] = "sentence\n"; ('\0' added automatically)
```

EXAMPLE

Draw the memory of `char name[] = "sentence\n";` and find its size.

Index	0	1	2	3	4	5	6	7	8	9
Char	s	e	n	t	e	n	c	e	\n	\0

8 letters + '\n' + '\0' = 10 characters, so `sizeof(name) = 10` bytes (verified by the program in the pointers topic below). '\n' is one character, not two.

Exam tip: Because C does no bounds checking, `char s[5] = "hello";` leaves no room for '\0'. Always allow one extra byte for the terminator.

Multidimensional arrays

Multidimensional array: an array with more than one subscript, declared with one pair of square brackets per dimension, e.g. `int a[3][5];` for a two-dimensional array.

Multidimensional array: ഒന്നിൽ കൂടുതൽ subscripts ഉള്ള array; ഓരോ dimension-നും ഓരോ ജോടി square brackets, ഉദാ: two-dimensional array-ക്ക് `int a[3][5];`.

A two-dimensional array needs two pairs of brackets, a three-dimensional one needs three, and so on. A 2-D array is best pictured as a table of rows and columns: the first subscript is the row, the second is the column.

മലയാളത്തിൽ

Theatre-ലെ seats ഓർക്കുക: row number-ഉം seat number-ഉം പറഞ്ഞാൽ കൃത്യമായ seat കിട്ടും. അതുപോലെ `a[1][3]` എന്നാൽ row 1, column 3. `int a[3][5]` എന്നാൽ 3 rows, 5 columns, ആകെ 15 elements.

```
data_type array_name[expr1][expr2] ... [exprn];
Bytes for a 2-D array = (size of 1st index) × (size of 2nd index) × sizeof(base type)
```

EXAMPLE 1

Show the layout of `int a[3][5];` (Figure 2.2).

	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>	<code>a[0][4]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>	<code>a[1][4]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>	<code>a[2][4]</code>

EXAMPLE 2

Memory needed for `int a[3][5];` (`int = 4 bytes`) and for `float m[10][20][2];` (`float = 4 bytes`)?

a: $3 \times 5 \times 4 = 60$ bytes.

m: $10 \times 20 \times 2 \times 4 = 1600$ bytes.

Exam tip: Check Your Progress 1, Q3 ("At maximum, an array can be two-dimensional") is **False**: C allows arrays of any number of dimensions. (Figure 2.2 in the unit labels the last row "Row 3"; it is Row 2.)

Arrays and pointers

Array name as pointer: an array name used without an index gives the address of its first element, so `a` and `&a[0]` are the same, and `a[i]` is equivalent to `*(a + i)`.

Array name as pointer: index ഇല്ലാതെ array-യുടെ പേര് ഉപയോഗിച്ചാൽ അതിന്റെ first element-ന്റെ address കിട്ടും; അതിനാൽ `a`-ഉം `&a[0]`-ഉം ഒന്നാണ്, `a[i]` എന്നത് `*(a + i)`-ന് തുല്യം.

Arrays and pointers are closely related. If `x = a;` then `x[5]` and `*(x + 5)` both refer to the sixth element of `a`. A pointer can therefore be indexed like an array. For a 2-D array stored row by row, element `a[j][k]` sits ($j \times \text{rowlength} + k$) elements after the start.

മലയാളത്തിൽ

Array-യുടെ പേര് ഒരു വീട്ടുപേര് പോലെയാണ്: അത് ആദ്യ വീടിന്റെ address തരും. അവിടുന്ന് 5 വീട് മുന്നോട്ട് പോയാൽ ആറാമത്തെ വീട്, അതാണ് `*(x + 5)`. 2-D array-ൽ row `j`-യിലെ column `k` കിട്ടാൻ, മുമ്പുള്ള `j` rows മുഴുവൻ ($j \times \text{rowlength}$) കടന്ന്, പിന്നെ `k` സ്ഥാനം മുന്നോട്ട് പോകണം.

```
p ≡ &p[0]
a[i] ≡ *(a + i)
a[j][k] ≡ *((base_type *)a + j × rowlength + k)
```

EXAMPLE

Verify these equivalences with a program (int a[3][5], so rowlength = 5).

```
#include <stdio.h>

int main(void)
{
    int a[3][5], *x, b[10];
    char name[] = "sentence\n";
    int i, j;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 5; j++)
            a[i][j] = 10 * i + j;

    x = b;
    x[5] = 100;          /* same as b[5] = 100    */
    *(x + 6) = 200;      /* same as b[6] = 200    */

    printf("sizeof(name) = %zu\n", sizeof(name));
    printf("b[5] = %d, b[6] = %d\n", b[5], b[6]);
    printf("a[0][4] = %d, *((int *)a + 4) = %d\n", a[0][4], *((int *)a + 4));
    printf("a[2][3] = %d, *((int *)a + 2*5 + 3) = %d\n", a[2][3], *((int *)a + 2 * 5 + 3));
    return 0;
}
```

```
sizeof(name) = 10
b[5] = 100, b[6] = 200
a[0][4] = 4, *((int *)a + 4) = 4
a[2][3] = 23, *((int *)a + 2*5 + 3) = 23
```

a[2][3] is $2 \times 5 + 3 = 13$ ints from the start, and both forms print 23.

Exam tip: The unit prints the general form as `*((base_type *)a + (j * rowlength)*k)`. That is a misprint: the offset is added, not multiplied. Correct: `*((base_type *)a + j * rowlength + k)`. Check: a[0][4] with the misprint gives offset 0, but the unit itself says a[0][4] is `*((int *)a + 4)`.

Sparse matrices: triangular and tridiagonal

Sparse matrix: a matrix in which a large number of the elements are zero.

Triangular matrix: a square matrix in which all elements above (lower triangular) or below (upper triangular) the main diagonal are zero.

Tridiagonal matrix: a square matrix in which all elements are zero except those on the main diagonal and the diagonals immediately above and below it.

Sparse matrix: elements-ൽ വലിയൊരു ഭാഗം പൂജ്യമായ matrix.

Triangular matrix: main diagonal-ന് മുകളിലുള്ള (lower triangular) അല്ലെങ്കിൽ താഴെയുള്ള (upper triangular) എല്ലാ elements-ഉം പൂജ്യമായ square matrix.

Tridiagonal matrix: main diagonal-ഉം അതിന് തൊട്ടു മുകളിലും താഴെയുമുള്ള diagonals-ഉം ഒഴികെ ബാക്കി എല്ലാ elements-ഉം പൂജ്യമായ square matrix.

Storing a sparse matrix in a full 2-D array wastes memory on zeros; in real applications this can be megabytes. Triangular and tridiagonal matrices are two common kinds of sparse matrices, and their non-zero elements can be stored compactly.

മലയാളത്തിൽ

ഒരു വലിയ hostel register-ൽ മിക്ക columns-ഉം ശൂന്യമാണെങ്കിൽ മുഴുവൻ പേജും സൂക്ഷിക്കുന്നത് പാഴാണ്; എഴുതിയ entries മാത്രം ഒരു ചെറിയ list-ൽ എഴുതിയാൽ മതി. Sparse matrix-ന്റെ ആശയം ഇതാണ്. Triangular matrix-ൽ diagonal-ന്റെ ഒരു വശം മുഴുവൻ പൂജ്യം; tridiagonal-ൽ നടുവിലെ മൂന്ന് diagonals മാത്രം.

$n \times n$ lower/upper triangular: at most $n(n + 1)/2$ non-zero elements

$n \times n$ tridiagonal: at most $3n - 2$ non-zero elements

Tridiagonal: $a[i][j] = 0$ whenever $|i - j| > 1$

EXAMPLE 1

The two matrices of Figure 2.3 (blanks are zeros).

(a) Lower triangular, 5×5 :

$$\begin{pmatrix} 4 & 0 & 0 & 0 & 0 \\ 3 & -5 & 0 & 0 & 0 \\ 1 & 0 & 6 & 0 & 0 \\ -7 & 8 & -1 & 3 & 0 \\ 5 & -2 & 0 & 2 & -8 \end{pmatrix}$$

(b) Tridiagonal, 7×7 :

$$\begin{pmatrix} 5 & -3 & 0 & 0 & 0 & 0 & 0 \\ 1 & 4 & 3 & 0 & 0 & 0 & 0 \\ 0 & 9 & -3 & 6 & 0 & 0 & 0 \\ 0 & 0 & 2 & 4 & -7 & 0 & 0 \\ 0 & 0 & 0 & 3 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & -5 & 8 \\ 0 & 0 & 0 & 0 & 0 & 3 & -1 \end{pmatrix}$$

EXAMPLE 2

How many elements need to be stored for (a) a 5×5 lower triangular matrix, (b) a 7×7 tridiagonal matrix? How many zeros are saved?

(a) $5 \times 6/2 = 15$ positions (rows hold 1, 2, 3, 4, 5). Saved: $25 - 15 = 10$.

(b) $3 \times 7 - 2 = 19$ positions (rows hold 2, 3, 3, 3, 3, 3, 2). Saved: $49 - 19 = 30$.

Exam tip: A band position can itself hold a zero (the fifth row of Figure 2.3(b) has a 0 in the band); the matrix is still tridiagonal because every element outside the band is zero.

3-tuple representation of a sparse matrix

3-tuple representation: a table in which the first row holds (number of rows, number of columns, number of non-zero elements) and each following row holds (row, column, value) of one non-zero element.

3-tuple representation: ആദ്യ row-ൽ (rows-ന്റെ എണ്ണം, columns-ന്റെ എണ്ണം, non-zero elements-ന്റെ എണ്ണം), തുടർന്നുള്ള ഓരോ row-ലും ഒരു non-zero element-ന്റെ (row, column, value) സൂക്ഷിക്കുന്ന table.

Only the non-zero elements are stored, row by row. For an $m \times n$ matrix with t non-zeros, the 3-tuple form needs $(t + 1) \times 3$ values instead of $m \times n$. It saves memory when $3(t + 1) < m \times n$.

മലയാളത്തിൽ

Class-ൽ absent ആയവരുടെ roll number മാത്രം എഴുതുന്നത് പോലെ: മുഴുവൻ matrix-ന് പകരം "ഏത് row, ഏത് column, എന്ത് value" എന്ന് non-zero elements-ന് മാത്രം എഴുതുന്നു. ആദ്യ വരി header ആണ്: matrix-ന്റെ size-ഉം non-zero എണ്ണവും.

Row 0: m, n, t

Rows 1 ... t: row index, column index, value (in row-major order)

Storage: $3(t + 1)$ values instead of $m \times n$

EXAMPLE 1

Write the 3-tuple form of the 7×6 matrix of Figure 2.4.

$$A = \begin{pmatrix} 0 & 0 & 0 & 5 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 9 & 0 \\ 0 & 3 & 0 & 2 & 0 & 0 \\ 1 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 0 & 0 \end{pmatrix}$$

Scan row by row (indices from 0). Non-zeros: 5, 4, 9, 3, 2, 1, 2, 8, so $t = 8$.

Row	Column	Value
7	6	8
0	3	5
1	1	4
2	4	9
3	1	3
3	3	2
4	0	1
4	2	2
6	2	8

Storage: $9 \times 3 = 27$ values instead of $7 \times 6 = 42$.

EXAMPLE 2 (PROGRAM 1.1, CORRECTED)

Read a matrix and print its 3-tuple form, including the header row.

```
/* Accepts a matrix and prints its 3-tuple representation */
#include <stdio.h>
#define MAX 10

int main(void)
{
    int a[MAX][MAX], rows, columns, i, j, count = 0;

    printf("Enter the order of the matrix (at most %d x %d):\n", MAX, MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX) {
        printf("Invalid order\n");
        return 1;
    }
    printf("Enter the elements of the matrix:\n");
    for (i = 0; i < rows; i++)
        for (j = 0; j < columns; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] != 0)
                count++;
        }

    printf("The 3-tuple representation of the matrix is:\n");
    printf("%d %d %d\n", rows, columns, count);
    for (i = 0; i < rows; i++)
        for (j = 0; j < columns; j++)
            if (a[i][j] != 0)
                printf("%d %d %d\n", i, j, a[i][j]);
    return 0;
}
```

```
Enter the order of the matrix (at most 10 x 10):
3 3
Enter the elements of the matrix:
1 2 3
0 1 0
0 0 4
The 3-tuple representation of the matrix is:
3 3 5
0 0 1
0 1 2
0 2 3
1 1 1
2 2 4
```

Given the matrix of Figure 2.4 as input, the same program prints exactly the table of Example 1 (7 6 8, then the eight triples).

Exam tip: Figure 2.5 in the unit starts with "7, 7, 9", but the text (and the matrix) says 7 rows, 6 columns, 8 non-zeros: the correct header is **7, 6, 8**. IGNOU's Program 1.1 also (i) uses `void main()`; standard C requires `int main(void)` returning 0, (ii) never prints the header row, (iii) prints 1-based positions $(i+1, j+1)$ while Figure 2.5 uses 0-based ones, and (iv) does not check that the order fits the array. The version above fixes all four.

Polynomials using arrays

Array representation of a polynomial: each non-zero term is stored as a (coefficient, exponent) pair in an array, with terms kept in decreasing order of exponent.

Array representation of a polynomial: ഓരോ non-zero term-ഉം ഒരു (coefficient, exponent) ജോഡിയായി array-ൽ സൂക്ഷിക്കുന്നു; terms exponent കറഞ്ഞു വരുന്ന ക്രമത്തിൽ.

A polynomial of degree n has at most $n + 1$ terms. Storing it as rows of (coefficient, exponent) in a 2-D array like `poly[6][2]` lets us add or multiply polynomials. To add, terms with the same exponent have their coefficients added; every other term is copied as it is.

മലയാളത്തിൽ

Polynomial കൂട്ടുന്നത് ചന്തയിൽ ഒരേ തരം സാധനങ്ങൾ ഒന്നിച്ച് എണ്ണുന്നതു പോലെയാണ്: x^2 ഉള്ളത് x^2 -നോടും x^4 ഉള്ളത് x^4 -നോടും മാത്രം കൂട്ടാം. ഓരോ term-ഉം array-യിൽ [coefficient, exponent] എന്ന രണ്ട് കള്ളികളിൽ ഇരിക്കും. ഒരേ exponent കണ്ടാൽ coefficients കൂട്ടുക; ഇല്ലെങ്കിൽ term അതേപടി result-ലേക്ക് പകർത്തുക.

`poly[i][0]` = coefficient of term i , `poly[i][1]` = exponent of term i

Addition: same exponent $\Rightarrow$ add coefficients; otherwise copy the term

Multiplication: $(a \cdot x^p)(b \cdot x^q) = ab \cdot x^{p+q}$, then combine equal exponents

EXAMPLE 1

Store $5x^4 + 2x^3 + 7x^2 + 10x - 8$ in an array.

i	0	1	2	3	4
poly[i][0] (coeff)	5	2	7	10	-8
poly[i][1] (exp)	4	3	2	1	0

EXAMPLE 2

Add $P = 5x^4 + 2x^3 + 7x^2 + 10x - 8$ and $Q = 3x^3 - 7x^2 + 4$ by merging (both in decreasing exponent order).

Compare	Action	Result term
P exp 4 vs Q exp 3	$4 > 3$, copy P term	$5x^4$
P exp 3 vs Q exp 3	equal, $2 + 3$	$5x^3$
P exp 2 vs Q exp 2	equal, $7 + (-7) = 0$	(dropped)
P exp 1 vs Q exp 0	$1 > 0$, copy P term	$10x$
P exp 0 vs Q exp 0	equal, $-8 + 4$	-4

$P + Q = 5x^4 + 5x^3 + 10x - 4$.

EXAMPLE 3 (PROGRAM 1.2, CORRECTED)

Add two polynomials entered as (coefficient, exponent) pairs, using the unit's three-pass method: matched exponents first, then unmatched terms of the first, then unmatched terms of the second.

```

/* Adds two polynomials; each term is stored as (coefficient, exponent) */
#include <stdio.h>

int main(void)
{
    int poly1[6][2], poly2[6][2], term1, term2, found, i, j;

    printf("Enter the number of terms in the first polynomial (at most 6):\n");
    scanf("%d", &term1);
    printf("Enter the number of terms in the second polynomial (at most 6):\n");
    scanf("%d", &term2);
    if (term1 < 1 || term1 > 6 || term2 < 1 || term2 > 6) {
        printf("Invalid number of terms\n");
        return 1;
    }
    printf("Enter the coefficient and exponent of each term of the first polynomial:\n");
    for (i = 0; i < term1; i++)
        scanf("%d %d", &poly1[i][0], &poly1[i][1]);
    printf("Enter the coefficient and exponent of each term of the second polynomial:\n");
    for (i = 0; i < term2; i++)
        scanf("%d %d", &poly2[i][0], &poly2[i][1]);

    printf("The resultant polynomial due to the addition of the input two polynomials:\n");

    /* 1. terms whose exponent appears in both: add coefficients */
    for (i = 0; i < term1; i++)
        for (j = 0; j < term2; j++)
            if (poly1[i][1] == poly2[j][1]) {
                printf("%d %d\n", poly1[i][0] + poly2[j][0], poly1[i][1]);
                break;
            }

    /* 2. terms of poly1 with no matching exponent in poly2 */
    for (i = 0; i < term1; i++) {
        found = 0;
        for (j = 0; j < term2; j++)
            if (poly1[i][1] == poly2[j][1])
                found = 1;
        if (!found)
            printf("%d %d\n", poly1[i][0], poly1[i][1]);
    }

    /* 3. terms of poly2 with no matching exponent in poly1 */
    for (i = 0; i < term2; i++) {
        found = 0;
        for (j = 0; j < term1; j++)
            if (poly2[i][1] == poly1[j][1])
                found = 1;
        if (!found)
            printf("%d %d\n", poly2[i][0], poly2[i][1]);
    }
}

```

```
    return 0;
}
```

```
Enter the number of terms in the first polynomial (at most 6):
5
Enter the number of terms in the second polynomial (at most 6):
4
Enter the coefficient and exponent of each term of the first polynomial:
1 2
2 4
3 6
1 8
5 7
Enter the coefficient and exponent of each term of the second polynomial:
5 2
6 9
3 6
5 7
The resultant polynomial due to the addition of the input two polynomials:
6 2
6 6
10 7
2 4
1 8
6 9
```

This is the unit's own test data: $(x^2 + 2x^4 + 3x^6 + x^8 + 5x^7) + (5x^2 + 6x^9 + 3x^6 + 5x^7) = 6x^2 + 6x^6 + 10x^7 + 2x^4 + x^8 + 6x^9$, matching IGNOU's output.

EXAMPLE 4

Multiply $(2x + 3)(x^2 - 1)$.

$2x \cdot x^2 = 2x^3$; $2x \cdot (-1) = -2x$; $3 \cdot x^2 = 3x^2$; $3 \cdot (-1) = -3$.

Result: $2x^3 + 3x^2 - 2x - 3$ (no equal exponents to combine).

Exam tip: IGNOU's Program 1.2 uses `void main()` (use `int main(void)` and `return 0;`), its last `printf` lacks `\n`, and its `match/proceed` flags are just a long way of writing "stop at the first match". The result it prints is correct but not in exponent order, and it does not drop terms whose coefficients add to 0. If the input is kept in decreasing exponent order, as the unit assumes, a single merge pass (Example 2) gives a sorted result in $O(m + n)$ time instead of $O(m \times n)$.

Representation of arrays in memory

Row major order: a 2-D array is stored row by row in consecutive memory locations.

Column major order: a 2-D array is stored column by column in consecutive memory locations.

Row major order: 2-D array row തിരിച്ച് ഒന്നിനു പിറകെ ഒന്നായി consecutive memory locations-ൽ സൂക്ഷിക്കുന്നു.

Column major order: 2-D array column തിരിച്ച് ഒന്നിനു പിറകെ ഒന്നായി consecutive memory locations-ൽ സൂക്ഷിക്കുന്നു.

Memory is a single line of addresses, but a 2-D array is a table, so the table has to be flattened into one line. There are two ways: lay the rows one after another (row major), or lay the columns one after another (column major). Which one is used is fixed by the programming language: C, C++, Java and Python/NumPy use row major; Fortran, MATLAB and R use column major.

മലയാളത്തിൽ

Memory ഒരു നീണ്ട ഒറ്റ വരി പോലെയാണ്; table അതിൽ വെക്കാൻ അതിനെ ഒരു വരിയാക്കണം. Bus-ൽ ആളുകളെ കയറ്റുന്നതു പോലെ: ഒന്നാം row-ലുള്ളവരെ മുഴുവൻ ആദ്യം, പിന്നെ രണ്ടാം row (row major); അല്ലെങ്കിൽ ഒന്നാം column-ലുള്ളവരെ മുഴുവൻ ആദ്യം, പിന്നെ രണ്ടാം column (column major). C row major ആണ് ഉപയോഗിക്കുന്നത്.

Array A with m rows and n columns, base address B, w bytes per element, lower bounds LB_r (rows), LB_c (columns):

Row major: $LOC(A[i][j]) = B + w \times [(i - LB_r) \times n + (j - LB_c)]$

Column major: $LOC(A[i][j]) = B + w \times [(j - LB_c) \times m + (i - LB_r)]$

In C ($LB = 0$): row major $LOC(a[i][j]) = B + w \times (i \times n + j)$

Exam tip: The unit says the choice "depends on the operating system". More precisely, it is decided by the language/compiler, not the OS: a C program stores arrays in row major order on every OS. Check Your Progress 2, Q1: an array can be stored either row wise (row major) or column wise (column major).

Row major representation

Row major representation: the first row of the array occupies the first set of memory locations reserved for it, the second row the next set, and so on.

Row major representation: array-യുടെ first row അതിനായി മാറ്റിവെച്ച memory-യിലെ ആദ്യ locations-ൽ, second row അടുത്ത locations-ൽ, അങ്ങനെ തുടർന്ന്.

To get row major order, take the second row and place it right after the last element of the first row, then the third row after that, and so on, until you have one single row. To reach $A[i][j]$ you skip i complete rows ($i \times n$ elements) and then j elements.

മലയാളത്തിൽ

Notebook-ൽ വായിക്കുന്നതു പോലെ: ഒരു വരി മുഴുവൻ ഇടത്തു നിന്ന് വലത്തോട്ട്, പിന്നെ അടുത്ത വരി. $A[i][j]$ കണ്ടുപിടിക്കാൻ മുമ്പുള്ള i rows മുഴുവൻ (ഓരോന്നിലും n elements) കടന്ന്, പിന്നെ j elements മുന്നോട്ട്.

Row 0 | Row 1 | Row 2 | ... | Row $m-1$

Offset of $A[i][j]$ (0-based) = $i \times n + j$

EXAMPLE 1

Write the row major order of the 3×4 array below.

$$\begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{pmatrix}$$

```
index:  0  1  2  3 | 4  5  6  7 | 8  9 10 11
        a  b  c  d | e  f  g  h | i  j  k  l
        --Row 0--- | --Row 1--- | --Row 2---
```

Row major: {a, b, c, d, e, f, g, h, i, j, k, l}. Check: $g = A[1][2]$ has offset $1 \times 4 + 2 = 6 \checkmark$.

EXAMPLE 2

`float A[10][20];` in C starts at address 2000 (float = 4 bytes). Find the address of $A[4][7]$.

$m = 10, n = 20, LB = 0$.

$LOC = 2000 + 4 \times (4 \times 20 + 7) = 2000 + 4 \times 87 = 2000 + 348 = \mathbf{2348}$.

EXAMPLE 3

A[1 ... 5][1 ... 6] has 2-byte elements and base address 100. Find LOC(A[3][4]) in row major order.

$m = 5, n = 6, LB_r = LB_c = 1$.

$LOC = 100 + 2 \times [(3 - 1) \times 6 + (4 - 1)] = 100 + 2 \times (12 + 3) = 100 + 30 = 130$.

EXAMPLE 4

Confirm that C uses row major order by printing byte offsets of int a[3][4].

```
#include <stdio.h>

int main(void)
{
    int a[3][4], i, j;
    char *base = (char *)a;

    printf("sizeof(a) = %zu\n", sizeof(a));
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++)
            printf("a[%d][%d]:+%2td ", i, j, (char *)&a[i][j] - base);
        printf("\n");
    }
    return 0;
}
```

```
sizeof(a) = 48
a[0][0]:+ 0  a[0][1]:+ 4  a[0][2]:+ 8  a[0][3]:+12
a[1][0]:+16  a[1][1]:+20  a[1][2]:+24  a[1][3]:+28
a[2][0]:+32  a[2][1]:+36  a[2][2]:+40  a[2][3]:+44
```

a[2][3] is at $4 \times (2 \times 4 + 3) = 44$, exactly the row major formula; rows follow one another in memory.

Exam tip: Check Your Progress 2, Q2: in row major representation the elements are stored row wise. In address questions, always subtract the lower bounds first; IGNOU often gives arrays like A[1 ... m][1 ... n].

Column major representation

Column major representation: the first column of the array occupies the first set of memory locations reserved for it, the second column the next set, and so on.

Column major representation: array-യുടെ first column അതിനായി മാറ്റിവെച്ച memory-യിലെ ആദ്യ locations-ൽ, second column അടുത്ത locations-ൽ, അങ്ങനെ തുടരും.

A quick way to get column major order: transpose the array, then read it in row major order. To reach $A[i][j]$ you skip j complete columns ($j \times m$ elements) and then i elements.

മലയാളത്തിൽ

ഇവിടെ വായന മുകളിൽ നിന്ന് താഴോട്ട്, column തിരിച്ച്: ആദ്യ column മുഴുവൻ, പിന്നെ രണ്ടാമത്തേത്. എളുപ്പ വഴി: matrix transpose ചെയ്ത് row major ആയി വായിക്കുക. $A[i][j]$ -ലേക്ക് എത്താൻ മുമ്പുള്ള j columns (ഓരോന്നിലും m elements) കടന്ന്, പിന്നെ i elements.

Col 0 | Col 1 | Col 2 | ... | Col $n-1$

Offset of $A[i][j]$ (0-based) = $j \times m + i$

EXAMPLE 1

Write the column major order of the same 3×4 array (a ... l).

```
index:  0  1  2 | 3  4  5 | 6  7  8 | 9 10 11
        a  e  i | b  f  j | c  g  k | d  h  l
        -Col 0- | -Col 1- | -Col 2- | -Col 3-
```

Column major: {a, e, i, b, f, j, c, g, k, d, h, l}. Check: $g = A[1][2]$ has offset $2 \times 3 + 1 = 7 \checkmark$.

EXAMPLE 2

For $A[10][20]$ of 4-byte floats at base 2000, find $LOC(A[4][7])$ in column major order and compare with row major.

$LOC = 2000 + 4 \times (7 \times 10 + 4) = 2000 + 4 \times 74 = \mathbf{2296}$.

Row major gave 2348, so the same element sits at different places in the two orders.

EXAMPLE 3

$A[1 \dots 5][1 \dots 6]$, $w = 2$, base 100: find $LOC(A[3][4])$ in column major order.

$LOC = 100 + 2 \times [(4 - 1) \times 5 + (3 - 1)] = 100 + 2 \times 17 = \mathbf{134}$ (row major gave 130).

EXAMPLE 4

A 2-D array $B[0 \dots 3][0 \dots 4]$ (4 rows, 5 columns) is stored column major from 1200 with 4-byte elements. Which element is at address 1256?

Offset = $(1256 - 1200)/4 = 14$ elements. In column major, offset = $j \times 4 + i$, so $j = 14 \div 4 = 3$ and $i = 14 \bmod 4 = 2$. The element is $\mathbf{B[2][3]}$.

Exam tip: The unit's column major answer ends "..., d, h, i"; that is a typo. The last element is l: {a, e, i, b, f, j, c, g, k, d, h, l}. Check Your Progress 2, Q3: in column major representation the elements are stored column wise.

Applications of arrays

Use of arrays: arrays suit problems where the number of items is fixed and known in advance and insertions and deletions are few, because elements can be accessed directly by index.

Use of arrays: items-ന്റെ എണ്ണം മുൻകൂട്ടി അറിയാവുന്നതും insertions, deletions കുറവുള്ളതുമായ problems-ന് array ഏറ്റവും യോജിച്ചതാണ്, കാരണം index വഴി elements നേരിട്ട് access ചെയ്യാം.

Arrays are simple and reliable: easy to traverse, search and sort, and any element is reached in constant time from its index. Their weaknesses are a fixed size decided in advance and costly insertion or deletion, which needs elements to be shifted (reshuffled), wasting time or leaving unused memory.

മലയാളത്തിൽ

Cinema hall-ലെ numbered seats പോലെ: seat number അറിഞ്ഞാൽ നേരെ അവിടെ എത്താം (fast access). പക്ഷേ നടുക്ക് ഒരാളെ കൂടി ഇരുത്തണമെങ്കിൽ എല്ലാവരും ഓരോ seat നീങ്ങണം; hall-ന്റെ size-ഉം മുൻകൂട്ടി നിശ്ചയിച്ചതാണ്. അതുകൊണ്ട് size fixed ആയ, മാറ്റങ്ങൾ കുറവുള്ള data-ക്ക് array നല്ലതാണ്.

Advantages	Disadvantages
Direct (random) access by index, $O(1)$	Size fixed at declaration
Easy to traverse, search and sort	Insertion/deletion need shifting, $O(n)$
Simple to declare and use	Unused slots waste memory
Contiguous storage, no pointer overhead	Only one data type (homogeneous)

EXAMPLE

An array holds {10, 20, 30, 40, 50} in $a[0 \dots 4]$ with room for 6. How many moves are needed to insert 15 at index 1?

Shift 50, 40, 30, 20 one place right (4 moves), then write 15 into $a[1]$. Result {10, 15, 20, 30, 40, 50}. Inserting at the front of n elements needs n shifts: this is why arrays suit data with few insertions and deletions.

Exam tip: Typical uses to quote: storing marks or matrices, lookup tables, strings, sparse matrix 3-tuples, polynomials, and as the underlying storage for stacks and queues.

Quick recap

- An array is a finite ordered set of homogeneous elements in contiguous memory.
- Memory = $\text{sizeof}(\text{type}) \times \text{number of elements}$; number of elements = $\text{UB} - \text{LB} + 1$ (C: $\text{LB} = 0$).
- 1-D address: $\text{LOC}(a[i]) = B + w(i - \text{LB})$.
- If initialized at declaration, the size may be omitted; uninitialized local arrays hold garbage.
- C does not check array bounds.
- A string is a char array ending in `'\0'`; "sentence\n" occupies 10 bytes.
- Multidimensional arrays use one `[ ]` per dimension; 2-D bytes = rows $\times$ columns $\times$ $\text{sizeof}(\text{type})$.
- Array name = address of first element: $a[i] \equiv *(a + i)$; $a[j][k] \equiv ((\text{type } *)a + j \times \text{rowlength} + k)$.
- Sparse matrix: many zeros. Triangular: $n(n + 1)/2$ non-zeros at most; tridiagonal: $3n - 2$.
- 3-tuple form: header (rows, cols, count), then (row, col, value) for each non-zero.
- Polynomials are stored as (coefficient, exponent) pairs; addition adds coefficients of equal exponents.
- Row major: $\text{LOC} = B + w[(i - \text{LB}_r)n + (j - \text{LB}_c)]$; C uses row major.
- Column major: $\text{LOC} = B + w[(j - \text{LB}_c)m + (i - \text{LB}_r)]$; Fortran uses column major.
- Arrays suit fixed-size data with few insertions/deletions.

Check yourself

Q. 1. If the array is _____ at the time of declaration, then the dimension of the array is optional.

Initialized. Example: `int b[] = {2, 3, 4, 5};` gets size 4.

Q. 2. A sparse matrix is a matrix having a good number of _____ elements. At maximum, can an array be two-dimensional?

Zero. The second statement is False: C supports arrays of any number of dimensions, e.g. `int a[2][3][4];`.

Q. 3. An array can be stored either _____ or _____. Which stores elements row wise, and which column wise?

Row wise (row major representation) or column wise (column major representation). Row major stores row wise; column major stores column wise.

Q. 4. How many bytes do `char s[] = "IGNOU";` and `double d[4][5];` (double = 8 bytes) occupy?

s: 5 letters + '\0' = 6 bytes.

d: $4 \times 5 \times 8 = 160$ bytes.

Q. 5. Give the 3-tuple representation of the 4×5 matrix with rows (0 0 3 0 0), (0 0 0 0 7), (0 0 0 0 0), (6 0 0 1 0).

Row	Column	Value
4	5	4
0	2	3
1	4	7
3	0	6
3	3	1

Storage: $5 \times 3 = 15$ values instead of 20.

Q. 6. Add $4x^5 + 3x^2 - 2$ and $-4x^5 + x^3 + 5x^2 + 7$ using the array method.

Exp 5: $4 + (-4) = 0$ (drop). Exp 3: only in second, copy x^3 . Exp 2: $3 + 5 = 8$. Exp 0: $-2 + 7 = 5$.

Result: $x^3 + 8x^2 + 5$, stored as (1, 3), (8, 2), (5, 0).

Q. 7. A[1 ... 10][1 ... 15] has 4-byte elements with base address 3000. Find LOC(A[5][8]) in row major and column major order.

$m = 10, n = 15$.

Row major: $3000 + 4 \times [(5 - 1) \times 15 + (8 - 1)] = 3000 + 4 \times 67 = \mathbf{3268}$.

Column major: $3000 + 4 \times [(8 - 1) \times 10 + (5 - 1)] = 3000 + 4 \times 74 = \mathbf{3296}$.

Q. 8. Write the row major and column major order of the 2×3 matrix with rows (1 2 3) and (4 5 6).

Row major: {1, 2, 3, 4, 5, 6}.

Column major: {1, 4, 2, 5, 3, 6}.

Q. 9. With `int *x, a[10]; x = a;`, which element does `*(x + 5) = 100;` change? For `int a[3][5]`, write `a[2][1]` in pointer form.

It sets `a[5]`, the sixth element.

`a[2][1] ≡ *((int *)a + 2 * 5 + 1) = *((int *)a + 11).`

Q. 10. Why is `void main()` in IGNOU's programs wrong, and when should arrays be avoided?

The C standard requires `main` to return `int` (`int main(void)` with `return 0;`) so the program reports its exit status to the OS; `void main` is non-standard. Arrays should be avoided when the size is not known in advance or when insertions and deletions are frequent, since these need shifting and waste time or memory.

UNIT 3

Lists

ഈ unit-ൽ List എന്ന Abstract Data Type, അത് array ഉപയോഗിച്ചും linked list ഉപയോഗിച്ചും C-യിൽ എങ്ങനെ implement ചെയ്യാം എന്ന് പഠിക്കുന്നു. Singly, doubly, circular linked list-കളിൽ insertion, deletion, traversal എന്നിവയും polynomial represent ചെയ്യുന്ന application-ഉം കാണാം.

Abstract Data Type: List

Array implementation of lists: insertion

Array implementation of lists: deletion

Singly linked list: nodes, head and NULL

Creating, traversing and counting a linked list

Inserting a node into a linked list

Deleting a node from a linked list

Doubly linked lists

Circularly linked lists: creation and traversal

Circular linked list: insertion and deletion

Application: representing a polynomial

Linked lists vs arrays: advantages and drawbacks

Abstract Data Type: List

Abstract Data Type (ADT): a collection of values together with a set of operations on those values, specified without any implementation details.

List (ADT): a finite sequence of elements of type T together with the operations create, update, delete, test for empty, test for full, find size and traverse.

Abstract Data Type (ADT): implementation details ഒന്നും പറയാതെ specify ചെയ്യുന്ന, ഒരു കൂട്ടം values-ഉം ആ values-ൽ ചെയ്യാവുന്ന operations-ഉം ചേർന്നതാണ്.

List (ADT): type T-യിലുള്ള elements-ന്റെ ഒരു finite sequence-ഉം അതിൽ ചെയ്യാവുന്ന create, update, delete, empty ആണോ എന്ന test, full ആണോ എന്ന test, size കണ്ടെത്തൽ, traverse എന്നീ operations-ഉം ചേർന്നതാണ്.

An ADT says what a data type holds and what you can do with it, not how it is stored. While defining an ADT we ignore space and time efficiency. Mathematically, "a type is a set, and the elements of the set are the values of that type". List elements can be integers, characters, reals or records (structures) mixing several types.

മലയാളത്തിൽ

ADT എന്നത് ഒരു hotel menu card പോലെയാണ്: എന്തൊക്കെ കിട്ടും (values), എന്തൊക്കെ order ചെയ്യാം (operations) എന്ന് മാത്രം പറയും; അടുക്കളയിൽ അത് എങ്ങനെ ഉണ്ടാക്കുന്നു (implementation) എന്ന് menu-വിൽ ഇല്ല. List ADT-യും അങ്ങനെ തന്നെ: create, insert, delete, traverse എന്നൊക്കെ ചെയ്യാമെന്ന് പറയും. അത് array വെച്ചാണോ pointer വെച്ചാണോ ചെയ്യുന്നത് എന്നത് പിന്നീടുള്ള കാര്യം.

List of type T = finite sequence $a_1, a_2, \dots, a_n$ ($n \geq 0$) with operations create, insert (update), delete, is_empty, is_full, size, traverse (visit)
Two implementations: contiguous (array) and linked (pointers + dynamic memory)

EXAMPLE: THE UNIT'S EMPLOYEE LIST

A company keeps a list of employee names that grows on recruitment and shrinks on retirement or termination. Specify it as an ADT.

Values: a sequence of employee records, e.g. `typedef struct { char name[20]; /* date of joining, ... */ } emp_list;`

Operations (only their headers matter at ADT level):

`create_emplist(emp_list *l):` make an empty list.

`insert_emplist(emp_list *l, char *name, int position):` put a valid name at position p.

`delete_emplist(emp_list *l, char *name):` remove that name.

`visit_emplist(emp_list *l):` visit (print) every name in order.

The bodies are written later in C, using either an array or a linked list. The user of the list does not need to know which.

Exam tip: "Define ADT List" is a common 2–3 mark question. Write the definition above, list the operations, and add one line: "It can be implemented contiguously (array) or as a linked list (pointers)."

Array implementation of lists: insertion

Contiguous (array) list: a list whose entries are stored in consecutive locations of an array, with a separate counter recording how many positions are occupied.

Contiguous (array) list: entries ഒരു array-യുടെ തുടർച്ചയായ locations-ൽ സൂക്ഷിക്കുകയും, എത്ര positions നിറഞ്ഞിട്ടുണ്ട് എന്ന് ഒരു separate counter-ൽ രേഖപ്പെടുത്തുകയും ചെയ്യുന്ന list.

We declare a structure holding the array and a count. Adding at the end is easy: store at index count and increment it. To insert at the beginning or in the middle, every element from the last one down to the insertion position must first move one place right, starting from the last so nothing is overwritten. So insertion costs $O(n)$ shifts in the worst case.

മലയാളത്തിൽ

ഒരു bench-ൽ നിരയായി ഇരിക്കുന്ന കുട്ടികളെ ഓർക്കുക. നടുവിൽ ഒരാളെ ഇരുത്തണമെങ്കിൽ അറ്റത്തുള്ള ആളിൽ നിന്ന് തുടങ്ങി ഓരോരുത്തരും ഒരു സീറ്റ് വലത്തോട്ട് നീങ്ങണം. Array-യിലും അങ്ങനെയാണ്: last element-ൽ നിന്ന് തുടങ്ങി insertion position വരെ എല്ലാം ഒന്ന് വലത്തോട്ട് shift ചെയ്യണം, എന്നിട്ട് count ഒന്ന് കൂട്ടണം. ആദ്യം മുതൽ shift ചെയ്താൽ values overwrite ആയിപ്പോകും.

n elements, insert at position r (1-based) $\Rightarrow$ shift n -th $\rightarrow$ $(n+1)$ -th, $(n-1)$ -th $\rightarrow$ n -th, ..., r -th $\rightarrow$ $(r+1)$ -th

Number of shifts = $n - r + 1$ (worst case $r = 1$: n shifts, $O(n)$)

Before inserting: overflow if count == MAX

EXAMPLE 1: THE UNIT'S TRACE

List (positions 1-7): 11 22 33 44 55 66 77. Insert 35 after 33, i.e. at position 4.

Step	Action	1	2	3	4	5	6	7	8
Start		11	22	33	44	55	66	77	
1	77 $\rightarrow$ pos 8	11	22	33	44	55	66	77	77
2	66 $\rightarrow$ pos 7	11	22	33	44	55	66	66	77
3	55 $\rightarrow$ pos 6	11	22	33	44	55	55	66	77
4	44 $\rightarrow$ pos 5	11	22	33	44	44	55	66	77
5	35 $\rightarrow$ pos 4, count = 8	11	22	33	35	44	55	66	77

Shifts = $n - r + 1 = 7 - 4 + 1 = 4$ (77, 66, 55, 44). $\checkmark$

EXAMPLE 2: THE INSERT FUNCTION (CORRECTED PROGRAM 3.1)

C arrays start at index 0, so here position is an index 0 ... count. Inserting 35 "at position 4" in the table above is index 3.

```
#define MAX 10

typedef struct
{
    int data[MAX];
    int count;
} list;

/* insert element at index position (0 .. count) */
void insert(list *start, int position, int element)
{
    int temp;
    if (start->count == MAX) {
        printf("List overflow\n");
        return;
    }
    if (position < 0 || position > start->count) {
        printf("Invalid position\n");
        return;
    }
    temp = start->count - 1;          /* last occupied index */
    while (temp >= position) {
        start->data[temp + 1] = start->data[temp];
        temp--;
    }
    start->data[position] = element;
    start->count++;
}
```

Trace for 11 22 33 44 55 66 77, insert(&l, 3, 35): temp starts at 6 (last index). It copies data[6] → data[7], data[5] → data[6], data[4] → data[5], data[3] → data[4], stops at temp = 2, then data[3] = 35 and count = 8. The complete program and its output are in the next topic.

Exam tip: IGNOU's Program 3.1 has bugs. (1) It starts shifting at temp = start->count, but the last element is at index count - 1, so the first copy moves an empty slot to data[count+1]. When the array already holds 9 of 10 items this writes past data[9]; when we compiled it, that write landed on count itself, and after the insertion the program reported count = 1 instead of 10. Start at count - 1. (2) There is no overflow check and no position check; add both. (3) void main() and fflush(stdin) are not standard C; use int main(void) and drop fflush(stdin). (4) Its create() also stores the terminating 0 into the array and never checks the array bound.

Array implementation of lists: deletion

Deletion in an array list: removing the element at position r by shifting every later element one place left and then decrementing the count.

Deletion in an array list: position r -ലെ element കളയാൻ അതിനു ശേഷമുള്ള എല്ലാ elements-ഉം ഒരു സ്ഥാനം ഇടത്തോട്ട് shift ചെയ്ത് count ഒന്ന് കുറയ്ക്കുന്നതാണ്.

Deleting the last element needs no shifting, just count - . Deleting from the beginning or middle leaves a hole, so $(r+1)$ -th moves to r -th, $(r+2)$ -th to $(r+1)$ -th, ..., n -th to $(n-1)$ -th. This time the shifting starts from the left, just after the deleted position. Worst case (deleting the first element) is $n - 1$ shifts, $O(n)$.

മലയാളത്തിൽ

Queue-ൽ നിന്ന് നടുവിലുള്ള ഒരാൾ പോയാൽ പിന്നിലുള്ളവർ ഓരോരുത്തരായി ഒരു step മുന്നോട്ട് വരും. ഇവിടെ shift ചെയ്യുന്നത് ഇടത്തോട്ടാണ്, അതുകൊണ്ട് deleted position-ന്റെ തൊട്ടടുത്ത element-ൽ നിന്നാണ് തുടങ്ങേണ്ടത്. അവസാനം count ഒന്ന് കുറയ്ക്കുക. Empty list-ൽ നിന്ന് delete ചെയ്യാൻ ശ്രമിക്കുന്നതിനെ underflow എന്ന് പറയും.

Delete position r (1-based) from n elements $\Rightarrow (r+1)$ -th $\rightarrow r$ -th, ..., n -th $\rightarrow (n-1)$ -th, count = $n - 1$

Number of shifts = $n - r$ (worst case $r = 1$: $n - 1$ shifts, $O(n)$)

Before deleting: underflow if count == 0

EXAMPLE 1: THE UNIT'S TRACE

Delete 44 (position 4) from 11 22 33 44 55 66 77.

Step	Action	1	2	3	4	5	6	7
Start		11	22	33	44	55	66	77
1	55 $\rightarrow$ pos 4	11	22	33	55	55	66	77
2	66 $\rightarrow$ pos 5	11	22	33	55	66	66	77
3	77 $\rightarrow$ pos 6, count = 6	11	22	33	55	66	77	

Shifts = $n - r = 7 - 4 = 3$. ✓

EXAMPLE 2: COMPLETE CORRECTED PROGRAM (PROGRAMS 3.1 + 3.2)

```

#include <stdio.h>
#define MAX 10

typedef struct
{
    int data[MAX];
    int count;
} list;

void traverse(list *start)
{
    int i;
    for (i = 0; i < start->count; i++)
        printf("%d ", start->data[i]);
    printf(" (count = %d)\n", start->count);
}

/* insert element at index position (0 .. count) */
void insert(list *start, int position, int element)
{
    int temp;
    if (start->count == MAX) {
        printf("List overflow\n");
        return;
    }
    if (position < 0 || position > start->count) {
        printf("Invalid position\n");
        return;
    }
    temp = start->count - 1;          /* last occupied index */
    while (temp >= position) {
        start->data[temp + 1] = start->data[temp];
        temp--;
    }
    start->data[position] = element;
    start->count++;
}

/* delete the element at index position (0 .. count-1) */
void delete_list(list *start, int position)
{
    int temp;
    if (position < 0 || position >= start->count) {
        printf("Invalid position\n");
        return;
    }
    printf("Deleting %d\n", start->data[position]);
    temp = position;
    while (temp < start->count - 1) {
        start->data[temp] = start->data[temp + 1];
        temp++;
    }
}

```

```

    }
    start->count--;
}

int main(void)
{
    list l = { {11, 22, 33, 44, 55, 66, 77}, 7 };
    traverse(&l);
    insert(&l, 3, 35);          /* after 33 */
    traverse(&l);
    delete_list(&l, 4);        /* remove 44 */
    traverse(&l);
    return 0;
}

```

Output:

```

11 22 33 44 55 66 77   (count = 7)
11 22 33 35 44 55 66 77 (count = 8)
Deleting 44
11 22 33 35 55 66 77   (count = 7)

```

After inserting 35, the value 44 sits at index 4, so `delete_list(&l, 4)` removes it.

Exam tip: IGNOU's `delete_list` (Program 3.2) prints `l->data[position]`, but no `l` exists inside the function; it must be `start->data[position]`. Its loop `while (temp <= start->count - 1)` also copies one slot too many (it reads `data[count]`, which is past the last element). The correct condition is `temp < count - 1`. It also never checks whether the position is valid.

Singly linked list: nodes, head and NULL

Linked list: a chain of structures (nodes), each holding data and a pointer that stores the address (link) of the next logical node in the list.

Head (start): the external pointer that holds the address of the first node; the last node's link is NULL.

Linked list: ഓരോന്നിലും data-യും list-ലെ അടുത്ത logical node-ന്റെ address (link) സൂക്ഷിക്കുന്ന ഒരു pointer-ഉം ഉള്ള structures-ന്റെ (nodes) ഒരു chain.

Head (start): ആദ്യത്തെ node-ന്റെ address സൂക്ഷിക്കുന്ന external pointer; last node-ന്റെ link NULL ആയിരിക്കും.

A linked list is like the bogies of a train: from the first bogie you follow each coupling to the next, and the bogie with no coupling is the last. Nodes need not sit next to each other in memory; each is allocated with `malloc()`

when needed. A node is a self-referential structure: it contains a pointer to a structure of its own type. An empty list is simply `head == NULL`.

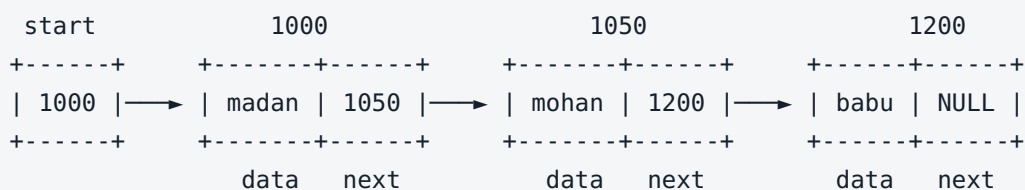
മലയാളത്തിൽ

Train-ന്റെ bogie-കൾ പോലെ ചിന്തിക്കുക: ആദ്യ bogie എവിടെയാണെന്ന് അറിഞ്ഞാൽ coupling പിടിച്ച് ഓരോന്നിലേക്കും പോകാം. Coupling ഇല്ലാത്ത bogie ആണ് അവസാനത്തേത്, അതായത് `next = NULL`. Array പോലെ memory-ൽ അടുത്തടുത്ത് ഇരിക്കണമെന്നില്ല; ഓരോ node-ഉം ആവശ്യം വരുമ്പോൾ `malloc()` കൊണ്ട് എവിടെയെങ്കിലും ഉണ്ടാക്കും. ആദ്യ node-ന്റെ address head-ൽ സൂക്ഷിക്കും; head നഷ്ടപ്പെട്ടാൽ list മുഴുവൻ നഷ്ടപ്പെട്ടു.

```
typedef struct node { int data; struct node *next; } list;
list *head = NULL; /* empty list */
Next node of p: p->next    Last node: p->next == NULL
```

EXAMPLE 1: FIGURE 3.1 IN MEMORY

Three nodes madan, mohan, babu stored at addresses 1000, 1050, 1200. Show the list.



start holds 1000 (madan's address). madan's next holds 1050 (mohan), mohan's next holds 1200 (babu), and babu's next is NULL, so babu is the last node. The addresses are not consecutive; only the links give the order.

EXAMPLE 2: PROGRAM 3.3, DECLARING AN EMPTY LIST

```
#include <stdio.h>

typedef struct node
{
    int data;
    struct node *next;
} list;

int main(void)
{
    list *head = NULL;          /* initialize list head to NULL */
    if (head == NULL)
        printf("The list is empty!\n");
    return 0;
}
```

Output:

```
The list is empty!
```

Exam tip: Draw every linked-list answer as boxes with arrows and show NULL at the end. It earns marks even when the code has small slips.

Creating, traversing and counting a linked list

Traversal: visiting every node of a linked list exactly once, starting from the head and following the next pointers until NULL is reached.

Traversal: head-ൽ നിന്ന് തുടങ്ങി next pointers പിന്തുടർന്ന് NULL എത്തുന്നതു വരെ linked list-ലെ ഓരോ node-ഉം ഒരു തവണ മാത്രം visit ചെയ്യുന്നത്.

To create a list, read values one by one. For each value, malloc a node, fill its data, set its next to NULL, and attach it after the current last node (or make it the head if the list is empty). Keeping a last pointer makes each attachment $O(1)$. To traverse, move a pointer $p = p \rightarrow \text{next}$ until it becomes NULL. Counting is a traversal that adds 1 per node; the unit writes it recursively: $\text{count}(\text{empty}) = 0$, $\text{count}(\text{list}) = 1 + \text{count}(\text{rest})$.

മലയാളത്തിൽ

Create ചെയ്യുമ്പോൾ ഓരോ value-ക്കും malloc() കൊണ്ട് ഒരു പുതിയ node ഉണ്ടാക്കി list-ന്റെ അറ്റത്ത് കൊള്ളത്തുന്നു. Traverse എന്നാൽ ഒരു വീട്ടിൽ നിന്ന് അടുത്ത വീട്ടിലേക്കുള്ള address നോക്കി post-man ഓരോ വീട്ടിലും കയറുന്നത് പോലെയാണ്: $p = p \rightarrow \text{next}$ ചെയ്ത് NULL കാണുന്നത് വരെ പോകും. Count ചെയ്യാൻ ഓരോ node-ലും ഒന്ന് കൂട്ടിയാൽ മതി; recursive ആയി "ബാക്കി list-ന്റെ count + 1" എന്നും എഴുതാം.

$\text{count}(\text{NULL}) = 0$

$\text{count}(p) = 1 + \text{count}(p \rightarrow \text{next})$

Creation, traversal and counting are all $O(n)$

EXAMPLE 1: CREATE, TRAVERSE, COUNT (CORRECTED PROGRAM 3.4)

```
typedef struct node
{
    int data;
    struct node *next;
} list;

/* read values until -1111; return the head of the new list */
list *create(void)
{
    list *head = NULL, *last = NULL, *n;
    int x;
    printf("Input elements, -1111 to stop: ");
    while (scanf("%d", &x) == 1 && x != -1111) {
        n = (list *) malloc(sizeof(list));
        n->data = x;
        n->next = NULL;
        if (head == NULL)
            head = n;          /* first node */
        else
            last->next = n;     /* link after the current last node */
        last = n;
    }
    printf("\n");
    return head;
}

void traverse(list *start)
{
    while (start != NULL) {
        printf("%d --> ", start->data);
        start = start->next;
    }
    printf("NULL\n");
}

int count(list *start)
{
    if (start == NULL)
        return 0;
    return 1 + count(start->next);
}
```

These functions, together with `find`, `insert_list` and `delete_list` in the next two topics, form one program; its main and output are in topic 7.

EXAMPLE 2: TRACE CREATION FOR INPUT 10 20 30 -1111

```
read 10: head → [10|NULL]                last = node 10
read 20: head → [10| ]→ [20|NULL]          last = node 20
read 30: head → [10| ]→ [20| ]→ [30|NULL]  last = node 30
read -1111: stop, return head
```

$\text{count}(\text{head}) = 1 + \text{count}(20\dots) = 1 + 1 + \text{count}(30) = 1 + 1 + 1 + \text{count}(\text{NULL}) = 3.$

Exam tip: IGNOU's Program 3.4 allocates the head node before reading any value, so the node that receives -1111 stays in the list as a dummy last node. Its traverse and count skip that node by testing $\text{start} \rightarrow \text{next} \neq \text{NULL}$, so the output is right, but it wastes a node and an empty list is never NULL. The version above ends with a real NULL. Also avoid `#define NULL 0` (it redefines the standard macro from `<stdio.h>`) and `void main()`.

Inserting a node into a linked list

Insertion in a linked list: allocating a new node and adjusting the links so that it appears at the required place (front, middle or end) without moving any other node.

Insertion in a linked list: ഒരു പുതിയ node allocate ചെയ്ത്, മറ്റൊരു node-ഉം നീക്കാതെ links മാത്രം മാറ്റി അതിനെ വേണ്ട സ്ഥലത്ത് (front, middle, end) ചേർക്കുന്നതാണ്.

Unit algorithm: (1) if the list is empty or the new element goes before the head, make it the new head; (2) if it goes after the last element, attach it at the end; (3) otherwise use `find` to get the address of the node before the insertion point and link the new node in. In Program 3.5 the new element is inserted **before** the node that holds key, so `find` returns the node whose next holds the key. Only two links change; nothing is shifted.

മലയാളത്തിൽ

ഒരു മാലയിൽ രണ്ട് മുത്തുകൾക്കിടയിൽ പുതിയൊരു മുത്ത് ചേർക്കുന്നത് പോലെ: ചരട് ഒരിടത്ത് പൊട്ടിച്ച് പുതിയ മുത്തിനെ രണ്ടറ്റത്തും ബന്ധിപ്പിക്കുന്നു, ബാക്കി മുത്തുകളൊന്നും നീങ്ങുന്നില്ല. ഇവിടെ ആദ്യം പുതിയ node-ന്റെ next-നെ key node-ലേക്ക് ചൂണ്ടിക്കാണാം, പിന്നെ മുന്തിയുള്ള node-ന്റെ next പുതിയ node-ലേക്ക്. ഈ order തെറ്റിച്ചാൽ list-ന്റെ ബാക്കി ഭാഗം നഷ്ടപ്പെടും.

Insert at front: $n \rightarrow \text{next} = \text{head}$; $\text{head} = n$;

Insert after node f: $n \rightarrow \text{next} = f \rightarrow \text{next}$; $f \rightarrow \text{next} = n$; (this order!)

Link changes $O(1)$; finding the position $O(n)$

EXAMPLE 1: FIGURES 3.2 AND 3.3 AS PICTURES

Insert 25 before key 30 in $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$.

```
Before:  head → [10| ] → [20| ] → [30| ] → [40|NULL]
          f = find(head, 30) (node before the key)
new:     n = [25| ]

Step 1:  n->next = f->next      [25| ] → [30| ]
Step 2:  f->next = n            [20| ] → [25| ]

After:   head → [10| ] → [20| ] → [25| ] → [30| ] → [40|NULL]
```

If Step 2 ran first, $f \rightarrow \text{next}$ would already point to n , so Step 1 would set $n \rightarrow \text{next} = n$: 30 and 40 would be lost and n would loop to itself.

EXAMPLE 2: FIND AND INSERT_LIST (CORRECTED PROGRAM 3.5)

```
/* return the node BEFORE the node holding key, or NULL */
list *find(list *start, int key)
{
    while (start != NULL && start->next != NULL) {
        if (start->next->data == key)
            return start;
        start = start->next;
    }
    return NULL;
}

/* insert element before the node holding key */
list *insert_list(list *start, int element, int key)
{
    list *n, *f;
    if (start != NULL && start->data == key) {    /* key is the first node */
        n = (list *) malloc(sizeof(list));
        n->data = element;
        n->next = start;
        start = n;                                /* new node becomes head */
    } else {
        f = find(start, key);
        if (f == NULL)
            printf("key %d is not found\n", key);
        else {
            n = (list *) malloc(sizeof(list));
            n->data = element;
            n->next = f->next;                    /* 1: new node -> key node */
            f->next = n;                          /* 2: previous node -> new node */
        }
    }
    return start;
}
```

Inserting 5 before key 10 (the head) takes the first branch, and the new node becomes the head, which is why the function returns start.

Exam tip: IGNOU's Program 3.5 has four bugs. (1) `mallo(sizeof(list))` is a typo for `malloc`. (2) The recursive `find` says `find(start->next, key)`; without `return`, so the result is lost (undefined behaviour); it must be `return find(start->next, key)`; (3) `find` reads `start->next->data` without checking that `start->next` is not `NULL`, so it crashes on a one-node list. (4) The heading says "insert at a specific position", but the code actually inserts before the key. The iterative `find` above fixes (2) and (3).

EXAMPLE 2: DELETE_LIST AND MAIN (CORRECTED PROGRAM 3.6)

```
/* delete the node holding key */
list *delete_list(list *start, int key)
{
    list *f, *temp;
    if (start == NULL)
        printf("list is empty\n");
    else if (start->data == key) {                /* key is the first node */
        temp = start->next;
        free(start);
        start = temp;
    } else {
        f = find(start, key);
        if (f == NULL)
            printf("key %d not found\n", key);
        else {
            temp = f->next->next;                /* node after the key node */
            free(f->next);
            f->next = temp;
        }
    }
    return start;
}

int main(void)
{
    list *head = create();
    traverse(head);
    printf("number of elements = %d\n", count(head));
    head = insert_list(head, 25, 30);    /* 25 before 30 */
    traverse(head);
    head = insert_list(head, 5, 10);    /* 5 before the head 10 */
    traverse(head);
    head = delete_list(head, 30);
    traverse(head);
    head = delete_list(head, 5);        /* delete the head */
    traverse(head);
    head = delete_list(head, 99);
    printf("number of elements = %d\n", count(head));
    return 0;
}
```

Run with input 10 20 30 40 -1111:

```

Input elements, -1111 to stop:
10 --> 20 --> 30 --> 40 --> NULL
number of elements = 4
10 --> 20 --> 25 --> 30 --> 40 --> NULL
5 --> 10 --> 20 --> 25 --> 30 --> 40 --> NULL
5 --> 10 --> 20 --> 25 --> 40 --> NULL
10 --> 20 --> 25 --> 40 --> NULL
key 99 not found
number of elements = 4

```

Exam tip: IGNOU's Program 3.6 main calls insert(head), but the function is named insert_list; it also uses curly quotes (" "), which do not compile. It reuses the buggy find from Program 3.5 and does not check for an empty list. All of these are fixed above.

Doubly linked lists

Doubly linked list: a linked list in which each node has three fields: a pointer to the left (previous) node, the data, and a pointer to the right (next) node.

Doubly linked list: ഓരോ node-യും മൂന്ന് fields ഉള്ള linked list: left (previous) node-ലേക്കുള്ള pointer, data, right (next) node-ലേക്കുള്ള pointer.

A singly linked list can be traversed only forwards. Links in both directions allow traversal both ways, and a node can be deleted when you only have a pointer to that node. The left link of the leftmost node and the right link of the rightmost node are NULL. The cost is one extra pointer per node and more links to update on every insertion and deletion.

മലയാളത്തിൽ

Singly linked list ഒരു one-way road പോലെയാണ്, മുന്നോട്ട് മാത്രം പോകാം. Doubly linked list two-way road ആണ്: ഓരോ node-നും മുമ്പത്തെ node-ലേക്കും (left) അടുത്ത node-ലേക്കും (right) link ഉണ്ട്. ആദ്യ node-ന്റെ left-ഉം അവസാന node-ന്റെ right-ഉം NULL. ഒരു extra pointer-ന്റെ memory കൂടുതൽ വേണം, insertion/deletion-ൽ നാല് links വരെ മാറ്റണം.

```
struct dl_list { int data; struct dl_list *right; struct dl_list *left; };
```

First node: left == NULL Last node: right == NULL

For every inner node p: p->right->left == p and p->left->right == p

NULL ← [| 10 |] ↔ [| 20 |] ↔ [| 30 |] → NULL
left data right

EXAMPLE 1: CREATE AND TRAVERSE BOTH WAYS (CORRECTED PROGRAM 3.7)

Unit algorithm: define a structure ELEMENT with data, left and right; allocate the first node with `malloc` and store its address in head with `head->left = NULL`; for each further value allocate a node, link it both ways, and repeat.

```

struct dl_list
{
    int data;
    struct dl_list *right;
    struct dl_list *left;
};

typedef struct dl_list dlist;

/* read values until -1111; return the head */
dlist *dl_create(void)
{
    dlist *head = NULL, *last = NULL, *n;
    int x;
    printf("Input the values, -1111 to come out: ");
    while (scanf("%d", &x) == 1 && x != -1111) {
        n = (dlist *) malloc(sizeof(dlist));
        n->data = x;
        n->right = NULL;
        n->left = last;          /* back link to the previous node */
        if (last == NULL)
            head = n;
        else
            last->right = n;    /* forward link from the previous node */
        last = n;
    }
    printf("\n");
    return head;
}

void traverse(dlist *start)
{
    dlist *last = NULL;
    printf("traversing the list using right pointer\n");
    while (start != NULL) {
        printf("%d = ", start->data);
        last = start;
        start = start->right;
    }
    printf("\ntraversing the list using left pointer\n");
    while (last != NULL) {
        printf("%d = ", last->data);
        last = last->left;
    }
    printf("\n");
}

```

EXAMPLE 2: INSERT AFTER A NODE AND DELETE A NODE

Insert 25 after node 20: four links change. Delete a node p: bypass it from both sides.

```
/* insert x after the node p (p != NULL) */
void insert_after(dlist *p, int x)
{
    dlist *n = (dlist *) malloc(sizeof(dlist));
    n->data = x;
    n->left = p;
    n->right = p->right;
    if (p->right != NULL)
        p->right->left = n;
    p->right = n;
}

/* delete node p; return the (possibly new) head */
dlist *dl_delete(dlist *head, dlist *p)
{
    if (p->left != NULL)
        p->left->right = p->right;
    else
        head = p->right;          /* p was the first node */
    if (p->right != NULL)
        p->right->left = p->left;
    free(p);
    return head;
}

int main(void)
{
    dlist *head = dl_create();
    printf("Created doubly linked list is as follows\n");
    traverse(head);
    insert_after(head->right, 25); /* after the 2nd node */
    traverse(head);
    head = dl_delete(head, head); /* delete the first node */
    traverse(head);
    return 0;
}
```

Run with input 10 20 30 -1111:

```

Input the values, -1111 to come out:
Created doubly linked list is as follows
traversing the list using right pointer
10 = 20 = 30 =
traversing the list using left pointer
30 = 20 = 10 =
traversing the list using right pointer
10 = 20 = 25 = 30 =
traversing the list using left pointer
30 = 25 = 20 = 10 =
traversing the list using right pointer
20 = 25 = 30 =
traversing the list using left pointer
30 = 25 = 20 =

```

Exam tip: IGNOU's Program 3.7 crashes. Its backward loop is `do { ... start = start->left; } while (start->right);`. After printing the first node, start becomes NULL and start->right dereferences NULL. When we compiled and ran it with input 1 2 3 -1111, it ended with a segmentation fault, not the printed output "3=2=1=". Loop while `(p != NULL)` instead. It also includes the non-standard `<malloc.h>`; use `<stdlib.h>`.

Circularly linked lists: creation and traversal

Circular linked list: a linked list in which the last node points back to the first node instead of containing NULL; an external pointer gives access to a starting node.

Circular linked list: last node-ൽ NULL-ന് പകരം first node-ലേക്കുള്ള pointer ഉള്ള linked list; ഒരു external pointer വഴിയാണ് starting node-ലേക്ക് എത്തുന്നത്.

No node holds NULL, so the chain itself has no natural first or last element; the external pointer (head) decides where we start. Operations are insertion, deletion and traversal. A traversal must stop when it comes back to the starting node, so it is written with `do ... while (p != head)`. Testing for NULL would loop forever.

മലയാളത്തിൽ

ഉത്സവത്തിന് കുട്ടികൾ കൈകോർത്ത് വട്ടത്തിൽ നിൽക്കുന്നത് പോലെ:

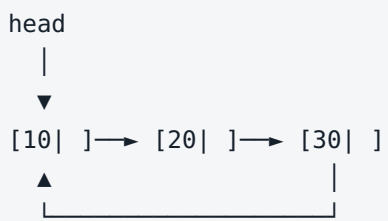
അവസാനത്തെ കുട്ടി ആദ്യത്തെ കുട്ടിയുടെ കൈ പിടിക്കുന്നു, അറ്റം എന്നൊന്നില്ല.

അതുകൊണ്ട് traverse ചെയ്യുമ്പോൾ NULL നോക്കിയിട്ട് കാര്യമില്ല; തുടങ്ങിയ node-ൽ തിരിച്ചെത്തിയാൽ നിർത്തണം. Round-robin CPU scheduling, multiplayer game-ലെ turns പോലുള്ളവയ്ക്ക് ഇത് ഉപകാരപ്പെടും.

Last node: `last->next == head`

Traversal: `p = head; do { visit(p); p = p->next; } while (p != head);`

One-node list: `head->next == head`



EXAMPLE 1: CREATE, TRAVERSE, COUNT (CORRECTED PROGRAM 3.8)

```

struct linked_list
{
    int data;
    struct linked_list *next;
};
typedef struct linked_list clist;

/* read values until -1111; the last node points back to the head */
clist *create_clist(void)
{
    clist *head = NULL, *last = NULL, *n;
    int x;
    printf("Input the elements, -1111 to stop: ");
    while (scanf("%d", &x) == 1 && x != -1111) {
        n = (clist *) malloc(sizeof(clist));
        n->data = x;
        if (head == NULL)
            head = n;
        else
            last->next = n;
        last = n;
        last->next = head;      /* close the circle */
    }
    printf("\n");
    return head;
}

void traverse(clist *head)
{
    clist *p = head;
    if (head == NULL) {
        printf("empty list\n");
        return;
    }
    do {
        printf("%d -> ", p->data);
        p = p->next;
    } while (p != head);
    printf("(back to %d)\n", head->data);
}

int count(clist *head)
{
    int c = 0;
    clist *p = head;
    if (head == NULL)
        return 0;
    do {
        c++;
        p = p->next;
    } while (p != head);
}

```

```
    return c;
}
```

The functions `find`, `insert_clist` and `delete_clist` and the output of the whole program are in the next topic.

EXAMPLE 2: WHY KEEP A POINTER TO THE LAST NODE?

If the external pointer `last` points to the last node, then the first node is `last->next`. Both ends are reachable in $O(1)$, so inserting at the front and at the end both take $O(1)$ time:

Front: `n->next = last->next; last->next = n;`

End: the same two lines followed by `last = n;`

Exam tip: IGNOU's Program 3.8 has the same dummy-node design as Program 3.4, but its traverse prints the last node when `start->next == s`, and that node is the dummy holding -1111. When we compiled and ran it with input 1 2 3 -1111, it printed "data is -1111" as a fourth element, while count said 3. It also prints pointers with %u; the correct format is %p with a (void *) cast.

Circular linked list: insertion and deletion

Insertion/deletion in a circular list: the same link changes as in a singly linked list, except that when the head node changes, the last node's link must be updated to the new head.

Circular list-ലെ insertion/deletion: singly linked list-ലെ അതേ link മാറ്റങ്ങൾ തന്നെ; പക്ഷേ head node മാറുമ്പോൾ last node-ന്റെ link പുതിയ head-ലേക്ക് update ചെയ്യണം.

Unit algorithm for insertion: if the list is empty or the element goes before the head, insert it as the start element; if it goes after the last element, insert it at the end and point it back to the start; otherwise locate the position with `find`. Deletion is the same as in a singly linked list. In a circular list the node before the head is the last node, so a `find` that goes all the way round naturally handles the head case too.

മലയാളത്തിൽ

വട്ടത്തിൽ നിൽക്കുന്ന കുട്ടികളിൽ ആദ്യത്തെ കുട്ടിക്ക് മുമ്പ് പുതിയൊരാൾ കയറിയാൽ, അവസാനത്തെ കുട്ടി ഇനി പിടിക്കേണ്ടത് പുതിയ ആളുടെ കൈയാണ്. അത് മറന്നാൽ വട്ടം പൊട്ടും. Code-ലും അതുപോലെ: head മാറുമ്പോൾ last->next പുതിയ head-ലേക്ക് മാറ്റണം. find function NULL അല്ല, തുടങ്ങിയ node-ൽ തിരിച്ചെത്തുന്നതാണ് നോക്കേണ്ടത്.

Insert before head: `last->next = n; n->next = head; head = n;`
Delete head: `last->next = head->next; free(head); head = last->next;`
Delete the only node: `head = NULL`

EXAMPLE 1: INSERT 5 BEFORE THE HEAD OF 10 → 20 → 30 → (10)

Before: head → [10|] → [20|] → [30|] → back to 10

`find(head, 10)` goes round and returns node 30 (the last node).

`n->next = 30->next (=10); 30->next = n; head = n`

After: head → [5|] → [10|] → [20|] → [30|] → back to 5

EXAMPLE 2: FIND, INSERT_CLIST, DELETE_CLIST AND MAIN (CORRECTED PROGRAMS 3.9 AND 3.10)

```

/* return the node BEFORE the node holding key, or NULL */
clist *find(clist *head, int key)
{
    clist *p = head;
    if (head == NULL)
        return NULL;
    do {
        if (p->next->data == key)
            return p;
        p = p->next;
    } while (p != head);
    return NULL;
}

/* insert x before the node holding key */
clist *insert_clist(clist *head, int x, int key)
{
    clist *n, *prev = find(head, key);
    if (prev == NULL) {
        printf("key %d is not found\n", key);
        return head;
    }
    n = (clist *) malloc(sizeof(clist));
    n->data = x;
    n->next = prev->next;
    prev->next = n;
    if (n->next == head)          /* key was the head: new node is the new head */
        head = n;
    return head;
}

/* delete the node holding key */
clist *delete_clist(clist *head, int key)
{
    clist *temp, *prev = find(head, key);
    if (prev == NULL) {
        printf("key %d not found\n", key);
        return head;
    }
    temp = prev->next;             /* node to delete */
    if (temp == prev)             /* it was the only node */
        head = NULL;
    else {
        prev->next = temp->next;
        if (temp == head)         /* deleting the head */
            head = temp->next;
    }
    free(temp);
    return head;
}

```

```

int main(void)
{
    clist *head = create_clist();
    traverse(head);
    printf("number of elements = %d\n", count(head));
    head = insert_clist(head, 5, 10);    /* before the head */
    traverse(head);
    head = insert_clist(head, 25, 30);
    traverse(head);
    head = delete_clist(head, 5);        /* delete the head */
    traverse(head);
    head = delete_clist(head, 30);
    traverse(head);
    head = insert_clist(head, 1, 99);
    printf("number of elements = %d\n", count(head));
    return 0;
}

```

Run with input 10 20 30 -1111:

```

Input the elements, -1111 to stop:
10 -> 20 -> 30 -> (back to 10)
number of elements = 3
5 -> 10 -> 20 -> 30 -> (back to 5)
5 -> 10 -> 20 -> 25 -> 30 -> (back to 5)
10 -> 20 -> 25 -> 30 -> (back to 10)
10 -> 20 -> 25 -> (back to 10)
key 99 is not found
number of elements = 3

```

Exam tip: IGNOU's Programs 3.9 and 3.10 are wrong for a circular list. (1) Their find stops on start->next->next == NULL, which never happens in a circular list, so a missing key recurses forever (stack overflow); it also lacks return on the recursive call. (2) Inserting before the head sets start = n but leaves the last node pointing to the old head, so the new node is outside the circle. (3) Deleting the head frees it while the last node (and the global s) still point to it, which leaves a dangling pointer. (4) The text says "insert after an existing element" but the code inserts before the key. The code above fixes all four.

Application: representing a polynomial

Polynomial as a linked list: each term is stored as a node holding its coefficient, its exponent and a link to the next term, usually in decreasing order of exponents.

Polynomial as a linked list: ഓരോ term-ഉം അതിന്റെ coefficient, exponent, അടുത്ത term-ലേക്കുള്ള link എന്നിവ ഉള്ള ഒരു node ആയി സൂക്ഷിക്കുന്നു, സാധാരണ exponents കുറഞ്ഞു വരുന്ന order-ൽ.

A polynomial is a sum of terms, each with a coefficient and an exponent; x is only a formal parameter and is not stored. A linked list suits polynomials because the number of terms varies and only non-zero terms need nodes. For $x^{1000} + 1$ an array indexed by exponent wastes 999 slots, while a list needs just two nodes.

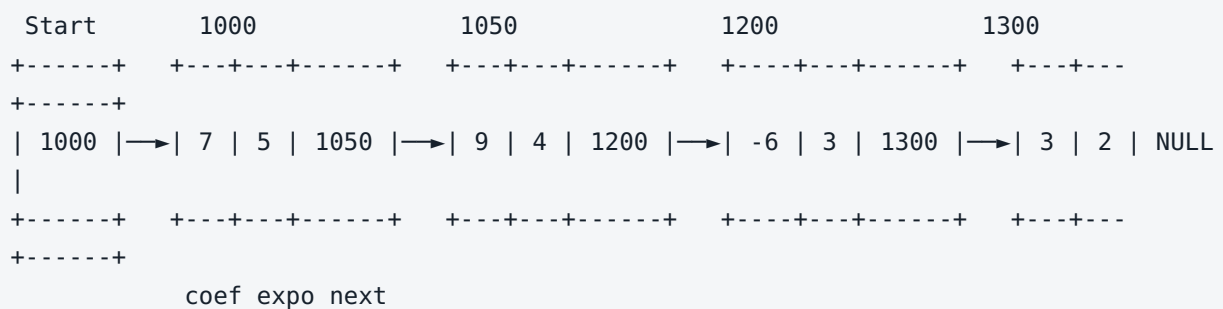
മലയാളത്തിൽ

Polynomial-ലെ ഓരോ term-ഉം ഒരു node: അതിൽ coefficient-ഉം exponent-ഉം. x എന്നത് ഒരു പേര് മാത്രം, അത് store ചെയ്യേണ്ടതില്ല. Zero ആയ terms-ന് node വേണ്ട, അതുകൊണ്ട് $x^{1000} + 1$ പോലുള്ളവയ്ക്ക് രണ്ടേ രണ്ട് node മതി. Array ആണെങ്കിൽ 1001 സ്ഥലം വേണ്ടിവരും.

```
struct link { int coef; int expo; struct link *next; };
```

$f(x) = 7x^5 + 9x^4 - 6x^3 + 3x^2 \Rightarrow 4 \text{ nodes: } (7,5) \rightarrow (9,4) \rightarrow (-6,3) \rightarrow (3,2) \rightarrow \text{NULL}$

EXAMPLE 1: FIGURE 3.10



EXAMPLE 2: BUILD AND PRINT THE POLYNOMIAL (CORRECTED PROGRAM 3.11)

```

#include <stdio.h>
#include <stdlib.h>

struct link
{
    int coef;                /* sign is part of the coefficient */
    int expo;
    struct link *next;
};
typedef struct link poly;

/* read coefficient-exponent pairs until exponent -1; return the head */
poly *create_poly(void)
{
    poly *head = NULL, *last = NULL, *n;
    int c, e;
    printf("Input coef expo pairs (expo -1 to stop): ");
    while (scanf("%d %d", &c, &e) == 2 && e != -1) {
        n = (poly *) malloc(sizeof(poly));
        n->coef = c;
        n->expo = e;
        n->next = NULL;
        if (head == NULL)
            head = n;
        else
            last->next = n;
        last = n;
    }
    printf("\n");
    return head;
}

void display(poly *start)
{
    int first = 1;
    while (start != NULL) {
        if (start->coef < 0)
            printf(first ? "-" : " - ");
        else if (!first)
            printf(" + ");
        printf("%dx^%d", abs(start->coef), start->expo);
        first = 0;
        start = start->next;
    }
    printf("\n");
}

int count_poly(poly *start)
{
    if (start == NULL)
        return 0;

```

```

    return 1 + count_poly(start->next);
}

int main(void)
{
    poly *head = create_poly();
    printf("Total nodes = %d\n", count_poly(head));
    display(head);
    return 0;
}

```

Input 7 5 9 4 -6 3 3 2 0 -1 (pairs coef expo; exponent -1 ends input). Output:

```

Input coef expo pairs (expo -1 to stop):
Total nodes = 4
7x^5 + 9x^4 - 6x^3 + 3x^2

```

EXAMPLE 3: ADDING TWO POLYNOMIAL LISTS BY HAND

Add $P = 5x^3 + 4x^2 + 2$ and $Q = 3x^4 - 4x^2 + 6x$.

Walk both lists together, always taking the larger exponent first:

exp 4: only Q $\rightarrow (3,4)$

exp 3: only P $\rightarrow (5,3)$

exp 2: both $\rightarrow 4 + (-4) = 0 \rightarrow$ no node

exp 1: only Q $\rightarrow (6,1)$

exp 0: only P $\rightarrow (2,0)$

Result: $3x^4 + 5x^3 + 6x + 2$, i.e. $(3,4) \rightarrow (5,3) \rightarrow (6,1) \rightarrow (2,0) \rightarrow \text{NULL}$.

Exam tip: IGNOU's Program 3.11 keeps the sign in a separate char field. That is redundant because coef is already a signed int. Its input also breaks: after `scanf("%d", ...)` the newline stays in the buffer (`fflush(stdin)` does not clear it in standard C), so leftover newlines are read by `getchar()` (as the choice) and by `scanf("%c")` (as the sign) instead of what the user typed. The version above stores the sign in coef and reads plain number pairs.

Linked lists vs arrays: advantages and drawbacks

Random access: reaching any element directly in $O(1)$ time from its index. Arrays support it; linked lists do not, because nodes are reached only by following links.

Random access: index ഉപയോഗിച്ച് ഏത് element-ലേക്കും $O(1)$ സമയത്തിൽ നേരിട്ട് എത്തുന്നത്. Array-യിൽ ഇത് സാധ്യമാണ്; linked list-ൽ links പിന്തുടർന്ന് മാത്രമേ nodes-ൽ എത്താൻ കഴിയൂ, അതുകൊണ്ട് സാധ്യമല്ല.

The advantage of linked lists is flexibility. Overflow is not a problem until the computer's memory is exhausted, there is no need to guess the size in advance, and insertion or deletion in the middle only changes links. The drawbacks: the links take extra space besides the data, and there is no random access, so reaching the k -th node means walking $k - 1$ links.

മലയാളത്തിൽ

Array ഒരു apartment block പോലെയാണ്: flat number അറിഞ്ഞാൽ നേരിട്ട് അവിടെ എത്താം (random access), പക്ഷേ flat-കളുടെ എണ്ണം ആദ്യമേ fix ആണ്. Linked list ഒരു treasure hunt പോലെയാണ്: ഓരോ സ്ഥലത്തും അടുത്ത സ്ഥലത്തിന്റെ clue മാത്രം; ഇഷ്ടം പോലെ സ്ഥലങ്ങൾ കൂട്ടാം, പക്ഷേ അഞ്ചാമത്തേതിൽ എത്താൻ ആദ്യ നാലിലും പോകണം.

Feature	Array (contiguous) list	Linked list
Size	Fixed in advance	Grows/shrinks at run time (malloc/free)
Overflow	When count == MAX	Only when memory is exhausted
Access k -th element	$O(1)$, random access	$O(k)$, sequential only
Insert/delete in middle	$O(n)$ shifts	$O(1)$ link changes once the position is known
Extra memory	None per element (but unused slots are wasted)	One pointer per node (two in a doubly linked list)
Storage	One contiguous block	Nodes can be anywhere in memory

EXAMPLE: CHOOSING A STRUCTURE

- (a) Marks of 60 students, looked up by roll number: **array** (fixed size, random access).
- (b) A playlist where songs are often inserted and removed in the middle: **linked list**.
- (c) Browser back/forward history: **doubly linked list** (move both ways).
- (d) Players taking turns in a loop: **circular linked list**.

Feature	Singly	Doubly	Circular (singly)
Links per node	1 (next)	2 (left, right)	1 (next)
Last node's link	NULL	right = NULL	points to first node
Traversal	Forward only	Both directions	Forward, can wrap round from any node
Stop condition	p == NULL	p == NULL	p == start again
Delete given only pointer to node	Needs predecessor (O(n) search)	O(1)	Needs predecessor (can go round)

Quick recap

- An ADT is a set of values plus operations, specified without implementation details. The List ADT has create, insert/update, delete, is_empty, is_full, size and traverse.
- A list can be implemented contiguously (array + count) or as a linked list (pointers + malloc).
- Array insertion at position r of n elements shifts n - r + 1 elements right, starting from the last element; deletion shifts n - r elements left. Both are O(n).
- Check overflow (count == MAX) before inserting and underflow (count == 0) before deleting.
- A linked-list node is a self-referential structure: data plus a pointer to the next node. The head holds the first node's address; the last node's next is NULL; an empty list has head == NULL.
- Traverse with p = p->next until NULL; count recursively as 1 + count(rest).
- Insert after node f: n->next = f->next; f->next = n; in this order. Inserting at the front changes head.
- Delete after node f: save f->next->next, free f->next, relink. Deleting the head moves head to the second node.
- A doubly linked list node has left, data and right; it allows two-way traversal and O(1) deletion of a given node, at the cost of an extra pointer.
- In a circular linked list the last node points to the first; traverse with do ... while (p != head), never test for NULL.
- When the head of a circular list changes, update the last node's link too.
- A polynomial is stored as a list of (coefficient, exponent) nodes; only non-zero terms need nodes.

- Linked lists: flexible size, cheap middle insert/delete. Drawbacks: extra link space and no random access.
- Several IGNOU programs in this unit have bugs (missing return in find, NULL dereference in Program 3.7, broken head handling in Programs 3.9 and 3.10); use the corrected versions.

Check yourself

Q. 1. Write a function to print the memory locations used to store the data in a singly linked list. (CYP 1)

```
void print_location(struct node *head)
{
    struct node *temp = head;
    while (temp != NULL) {
        printf("data %d is stored at %p\n", temp->data, (void *) temp);
        temp = temp->next;
    }
}
```

Sample output for a 3-node list (the addresses will differ on every run):

```
data 10 is stored at 0x7ffd7cf06cf0
data 20 is stored at 0x7ffd7cf06ce0
data 30 is stored at 0x7ffd7cf06cd0
```

The unit's answer uses an undeclared temp, prints pointers with %u (use %p), and dereferences head without checking, so it crashes on an empty list. The loop above handles all these.

Q. 2. Can we use a doubly linked list as a circular linked list? If yes, explain. (CYP 2)

Yes. Set the right link of the last node to the first node and the left link of the first node to the last node. The result is a **circular doubly linked list**. No link is NULL; from the head, head->left gives the last node in O(1), and it can be traversed round in either direction, stopping when the start node is reached again.



Q. 3. Write the differences between a doubly linked list and a circular linked list. (CYP 3)

Doubly linked list	Circular linked list
Each node has two pointers (left and right) plus data.	Each node has one pointer (next) plus data (singly circular).
First node's left and last node's right are NULL.	No NULL link; the last node points to the first.
Can be traversed forward and backward.	Traversed in one direction, but can wrap round from any node back to itself.
Traversal stops at NULL.	Traversal stops when the starting node is reached again.
More memory per node; insert/delete updates up to 4 links.	Less memory per node; insert/delete updates 2 links (plus the last node if the head changes).
A given node can be deleted in $O(1)$.	Deleting a given node needs its predecessor (found by going round).

Q. 4. Write a program to count the number of items stored in a singly linked list. (CYP 4)

```
int count_items(struct node *head)
{
    int count = 0;
    struct node *temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

For the list $10 \rightarrow 20 \rightarrow 30$ this returns 3, and for an empty list (NULL) it returns 0. The unit's answer never advances temp (the line `temp = temp->next;` is missing), so its loop runs forever. It also misspells `printf` as `pintf`, uses an undeclared temp, and crashes on an empty list. The recursive count from topic 5 is an equally good answer.

Q. 5. Write a function to check the overflow condition of a list represented by an array. (CYP 5)

```
int is_overflow(int count, int max_size)
{
    return count == max_size;
}
```

The list is full when the number of stored elements equals the array size, so another insertion would overflow. The unit's answer compares `last_element_position == max_size`, which is correct only if positions are counted from 1. With C's 0-based indices the last usable index is `max_size - 1`, so test `count == max_size` (or `last_index == max_size - 1`).

Q. 6. An array list holds 12 elements. How many shifts are needed to (a) insert at position 5, (b) delete position 5, (c) insert at the end?

- (a) $n - r + 1 = 12 - 5 + 1 = 8$ shifts (elements 12 down to 5 move right).
- (b) $n - r = 12 - 5 = 7$ shifts (elements 6 to 12 move left).
- (c) 0 shifts: store at position 13 and increment count (if the array is not full).

Q. 7. To insert node *n* after node *f*, a student writes `f->next = n; n->next = f->next;`. What goes wrong?

After the first statement, `f->next` is already `n`, so the second sets `n->next = n`. The node points to itself and every node that followed `f` is lost (memory leak, broken list). The correct order is `n->next = f->next;`
`f->next = n;`.

Q. 8. For the singly linked list $4 \rightarrow 8 \rightarrow 15 \rightarrow 16 \rightarrow 23$, show the list after (i) inserting 42 before 15, (ii) deleting 4, (iii) deleting 23.

- (i) find returns node 8; `n->next = 15`, `8->next = n`: $4 \rightarrow 8 \rightarrow 42 \rightarrow 15 \rightarrow 16 \rightarrow 23$.
- (ii) 4 is the head: `head = node 8`, free node 4: $8 \rightarrow 42 \rightarrow 15 \rightarrow 16 \rightarrow 23$.
- (iii) find returns node 16; `temp = 16->next->next = NULL`, free node 23, `16->next = NULL`: $8 \rightarrow 42 \rightarrow 15 \rightarrow 16 \rightarrow \text{NULL}$.

Q. 9. Draw the linked-list representation of $p(x) = 6x^4 - 2x^2 + 5$ and state how many nodes it needs.

```
head → [ 6 | 4 | ] → [ -2 | 2 | ] → [ 5 | 0 | NULL ]
        coef expo
```

3 nodes: only the non-zero terms are stored. The constant 5 is the term $5x^0$.

Q. 10. In a circular list, why is `while (p != NULL)` a wrong traversal condition, and what should be used?

No node in a circular list has a NULL link, so `p` never becomes NULL and the loop runs forever. Remember the starting node and stop when you get back to it: `p = head; do { visit(p); p = p->next; } while (p != head);` (after checking that head is not NULL).

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.